

Bachelor's and master's projects

- Automatic preprocessing & quality estimation of wearable EEG
- Finding embedding spaces for sleep data
- Clinical chatbots
- Activity detection in wearables dataset

All projects are based on the material covered in this course.

Study course

Some students have approached me about a 5 ECTS 'study course' on MLops – meaning best practices for machine learning projects.

The study course would focus on following material from a course offered at DTU: https://skaftenicki.github.io/dtu_mlops/

Study courses are student driven. That means a group of students come together to create a course about a topic that they think is missing. The students then take turns teaching each other. To make sure the students can get credit for the course at the end, a lecturer 'oversees' the sessions and participates to some extent (that would be me). But much less than in a regular course.

Exam would be a written report without grades.

If you would like to be part of this, send me an email, and I'll make sure to connect all interested parties to flesh out a plan.

The study course would take place next semester.

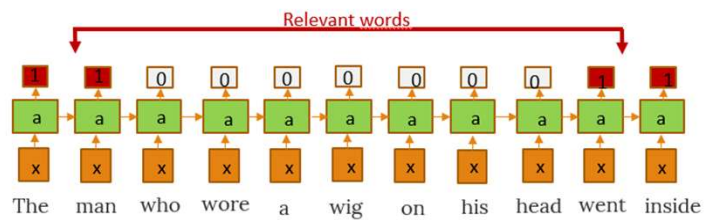
Deep Learning

NLP – Natural Language Processing

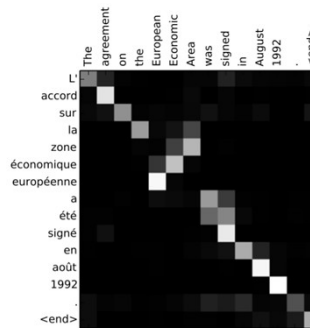
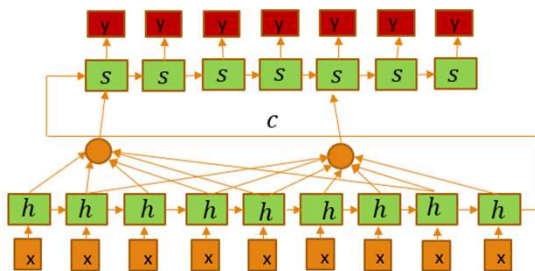


Last week

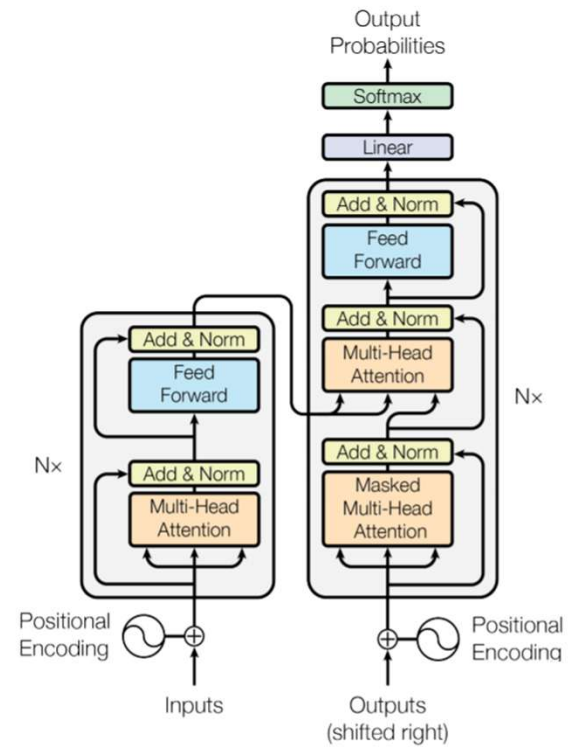
Sequence models:



Encoder, decoder & attention:

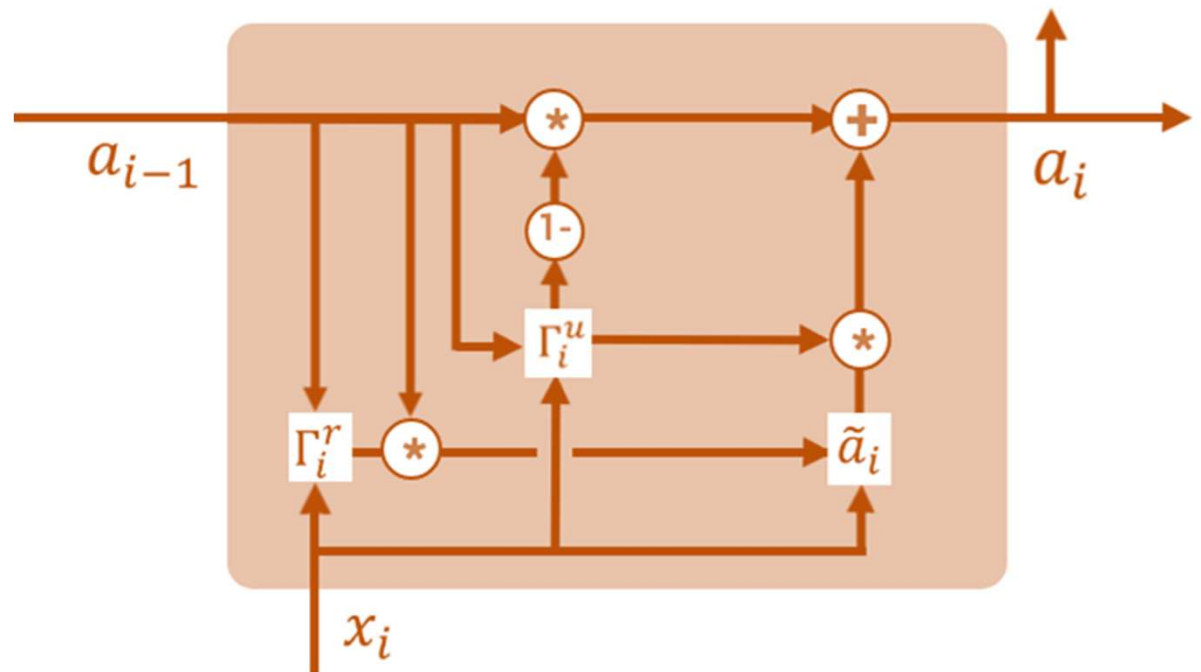


Transformers:



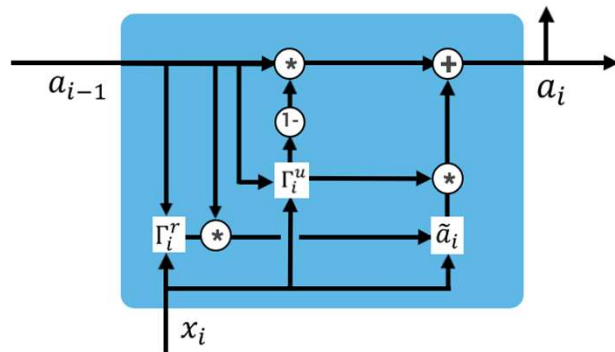
Quiz

What is this?



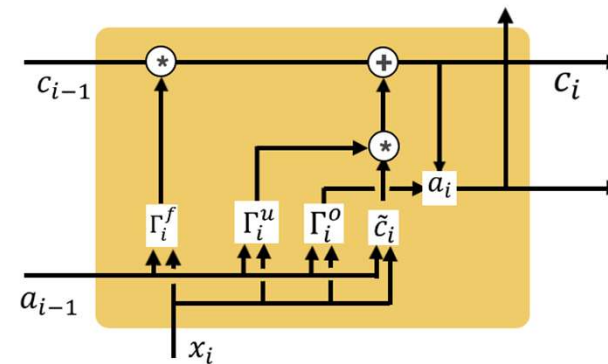
GRU – Gated Recurrent Unit:

$$\begin{aligned}\Gamma_i^r &= \sigma(W_{rx}x_i + W_{ra}a_{i-1} + b_r) : \text{reset gate} \\ \Gamma_i^u &= \sigma(W_{ux}x_i + W_{ua}a_{i-1} + b_u) : \text{update gate} \\ \tilde{a}_i &= \tanh(W_{ax}x_i + W_{aa}(\Gamma_i^r * a_{i-1}) + b_a) : \text{candidate} \\ a_i &= (1 - \Gamma_i^u) * a_{i-1} + \Gamma_i^u * \tilde{a}_i\end{aligned}$$



LSTM – Long Short Term Memory:

$$\begin{aligned}\Gamma_i^o &= \sigma(W_{ox}x_i + W_{oa}a_{i-1} + b_o) : \text{output gate} \\ \Gamma_i^u &= \sigma(W_{ux}x_i + W_{ua}a_{i-1} + b_u) : \text{update gate} \\ \Gamma_i^f &= \sigma(W_{fx}x_i + W_{fa}a_{i-1} + b_f) : \text{forget gate} \\ \tilde{c}_i &= \tanh(W_{cx}x_i + W_{ca}a_{i-1} + b_c) : \text{candidate} \\ c_i &= \Gamma_i^f * c_{i-1} + \Gamma_i^u * \tilde{c}_i : \text{cell state} \\ a_i &= \Gamma_i^o * \tanh(c_i)\end{aligned}$$



Natural Language Processing

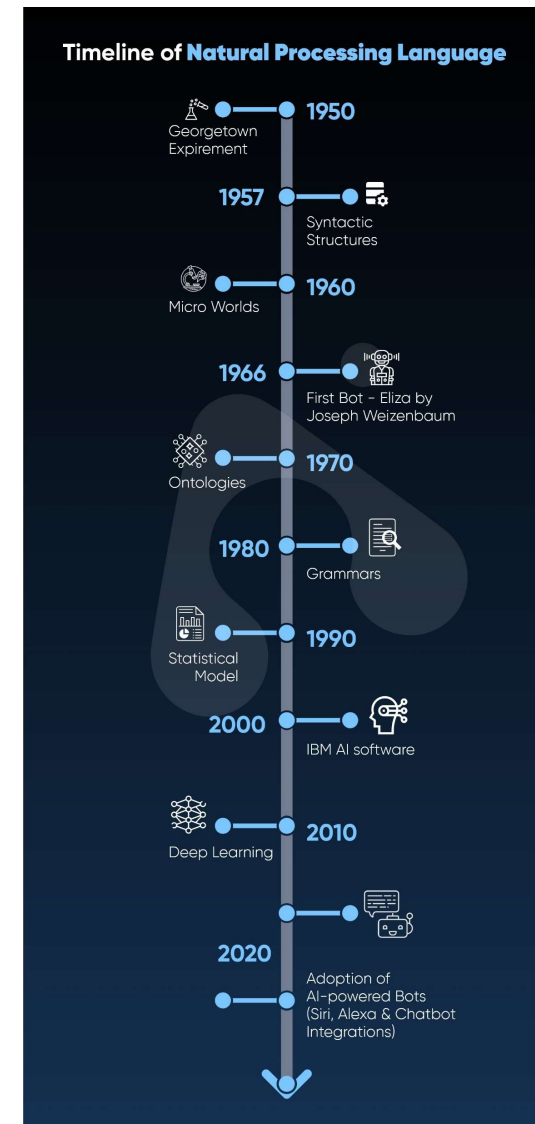
NLP is considerably older than deep learning.

It may be said to have started with Alan Turing and the 'Turing test' in 1950, or it may even predate computers, by going all the way back to Descartes.

However, modern NLP is often dated back to a paper by Bengio et al in 2003, which presented a 'Neural Probabilistic Language Model' (NLM).

We start with Bengio, and then try to encompass the (many) major innovations of the past 20ish years.

Sources:
https://en.wikipedia.org/wiki/History_of_natural_language_processing
<https://ruder.io/a-review-of-the-recent-history-of-nlp/>
<https://www.appventurez.com/blog/beginners-guide-to-natural-language-processing-nlp/amp/>



Focus today:

Yes:

History, some NLP-specific details, what is the principle of modern language models

No:

How to train your own Large Language Model



Natural Language Processing

Last week we touched upon some uses of NLP:

Sentiment
analysis

Advanced
spellcheckers

Automatic
translation

Speech-to-
text

Image
captioning

Text
generation

Chatbots

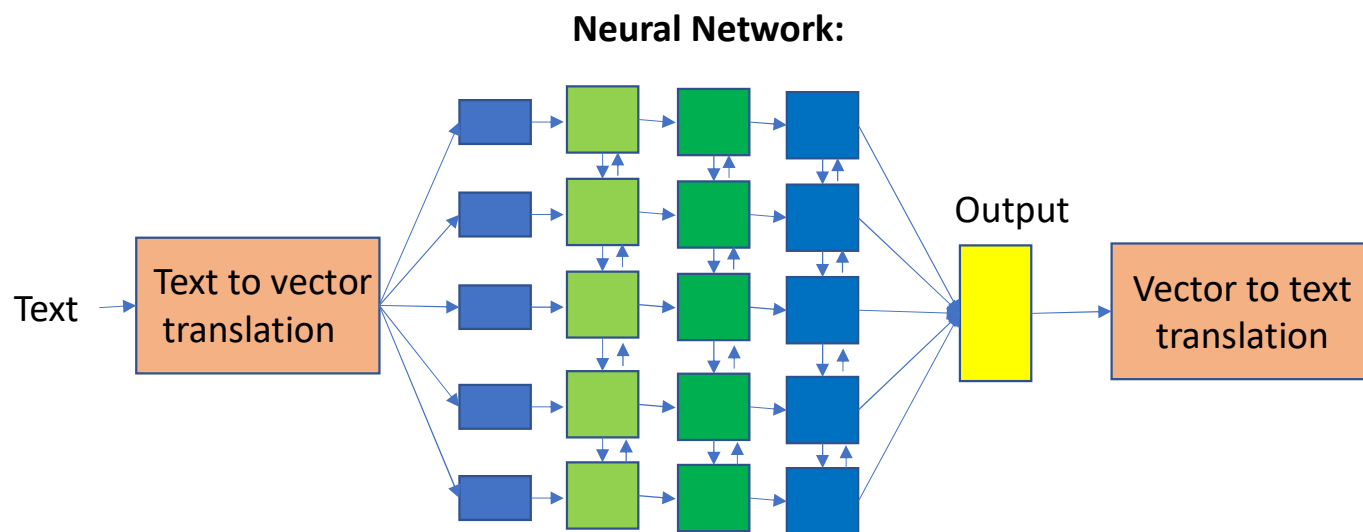
Text-to-
image

Sources: https://en.wikipedia.org/wiki/History_of_natural_language_processing, <https://ruder.io/a-review-of-the-recent-history-of-nlp/>

This week

To a large extent, modern NLP builds on the tools we introduced last week.

However, there are some details specific to NLP, which we go through today. In particular, we will discuss how to actually input text into a neural network:





How are we going to
do that?

Plan for today:

Oldschool: Bag of words

Newer: Embeddings!

- Word2Vec
- GloVe
- ELMo, BERT & ERNIE

Newest: Transformers



Old school: Bag of words & one-hot encoded vectors

The simplest way to translate a piece of text into numerical data is to sort each possible word into a dictionary, and then count how many of each word we have:

Word	Index:
A	1
Aardvark	2
...	
Rhinoceros	78351
Rhinestone	78352
...	
Xylophone	100000

We can then assign each word in a sentence an index:

A rhinoceros sat and played the xylophone
1 78351 82004 156 65007 87654 100000

We can also represent each word as a one-hot-encoded vector:

$$\mathbf{A} \rightarrow o_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad \mathbf{xylophone} \rightarrow o_{100000} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

This leads to the 'bag of words', which is simply a sum of all vectors:

$$bow = \sum_i o_{j_i}$$



Bag of words & one-hot encoded vectors:



This approach has a number of issues:

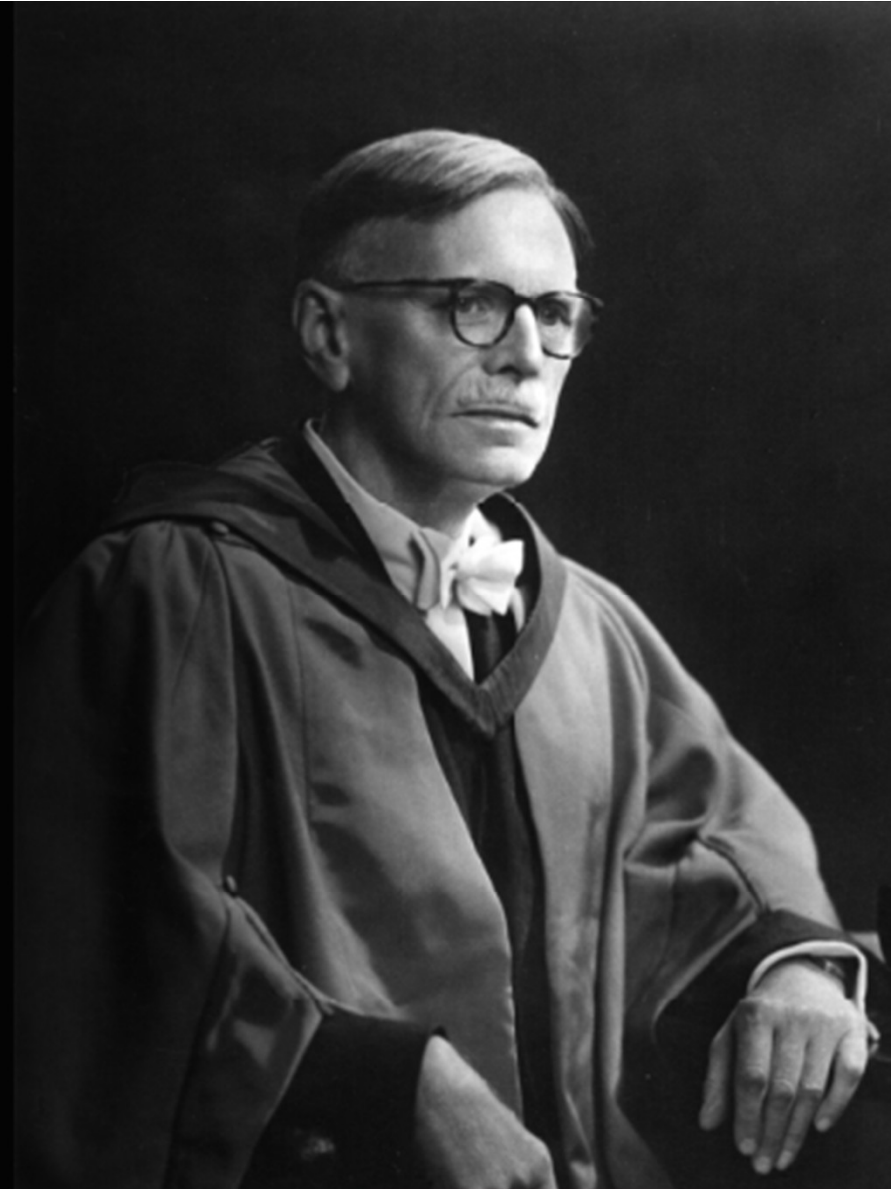
- It does not work with unknown words
- Bag-of-words has no information on the ordering of words
- One-hot-encoded vectors need to be as long as the vocabulary size (easily +100 000 for any serious system)
- One-hot-encoded vectors do not have a good similarity measure.



Word embeddings

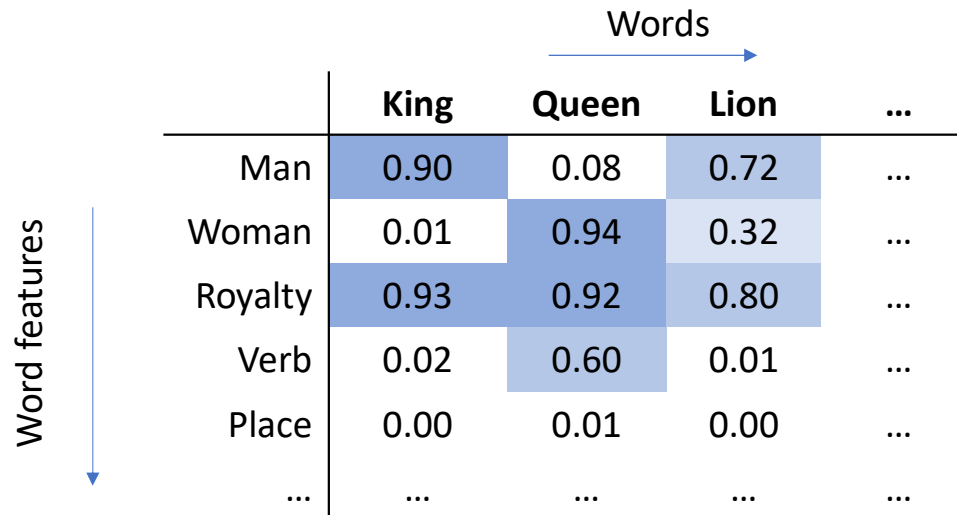
"You shall know a word by the
company it keeps"

- John Rupert Firth 1957



Word embeddings

Imagine an N-dimensional space where each dimension quantifies some quality of a word:



A diagram illustrating word embeddings. A vertical blue arrow on the left points downwards and is labeled "Word features". A horizontal blue arrow at the top points to the right and is labeled "Words".

	Words			
	King	Queen	Lion	...
Man	0.90	0.08	0.72	...
Woman	0.01	0.94	0.32	...
Royalty	0.93	0.92	0.80	...
Verb	0.02	0.60	0.01	...
Place	0.00	0.01	0.00	...
...	...	...	...	...

(Assume $N=300$)



Word embeddings

Benefits of word embeddings:



Can be learned unsupervised
(reduces issue with unknown words)



Gives a metric between words



Reduces input dimension

Generating a word embedding

The trick is going to be in predicting a (target) word based on its context:

The quick brown fox jumps over the lazy dog

Context Target

By clever design, we can create a model that makes an embedding as part of solving this problem.

Generating a word embedding

Embeddings were introduced in Bengio et al 2003, with the following general approach:

Given a context, c , N words long, predict a missing target word, t . Achieve this by minimizing the negative conditional probability $p(t|c, \theta)$ for all contexts & targets:

$$L(\theta) = - \prod_i^N P(t_i | c_i, \theta), \theta: \text{model parameters}$$

To make the calculations more stable, we focus on $\log(L)$:

$$\log(L(\theta)) = - \sum_i^N \log(P(t_i | c_i, \theta))$$

Word context

Consider the word 'fox' in the sentence below, and a 4-word context around it:

The quick brown fox jumps over the lazy dog.

In NLP, n consecutive items (words, letters, phonemes) from a piece of text or speech is called an n -gram. '2-grams' are called 'bigrams'.

Asymmetric contexts can also be relevant, such as the one we use in text generation:

The quick brown fox jumps over the lazy dog.

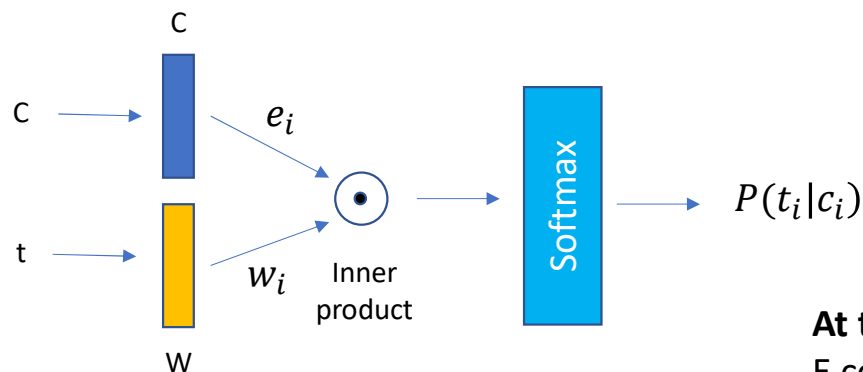


Generating a word embedding

In the original paper, the $P(t_i|c_i, \theta)$ were calculated by a multilayer neural network, with a softmax layer at the end. However, in 2013, Mikolov et al showed that a very simple 'network' is sufficient:

$$P(t_i|c_i, \{W, E\}) = \frac{e^{e_i \cdot w_i}}{\sum_j e^{e_j \cdot w_i}}, e_i = E o_{c_i}, w_i = W o_{t_i}$$

Considered as a network (with only E and W as parameters):



At the beginning of training:

E and W are initialized randomly as the 'weights' of the model.

At the end of training:

E contains the embedding for the vocabulary. E and W are of dimensions $d \times N$, where N is the size of the vocabulary and d is often around 300.

Generating a word embedding

Under this convention, we still need to decide exactly how a context is represented as a vector.

In the 2013 paper were presented two approaches: '**Continuous-bag-of-words**' (CBOW) & '**Skip-gram**' :

CBOW:

We pick the surrounding n words, and make an average encoding vector:

$$o_{c_i} = \frac{1}{n} \sum_{j \in c_i} o_j$$

This way, o_{c_i} becomes the average of the individual embedding vectors.

Generating a word embedding

Skip-gram:

Instead of averaging within the context, we may simply pick a random word, and try to infer the target from that:

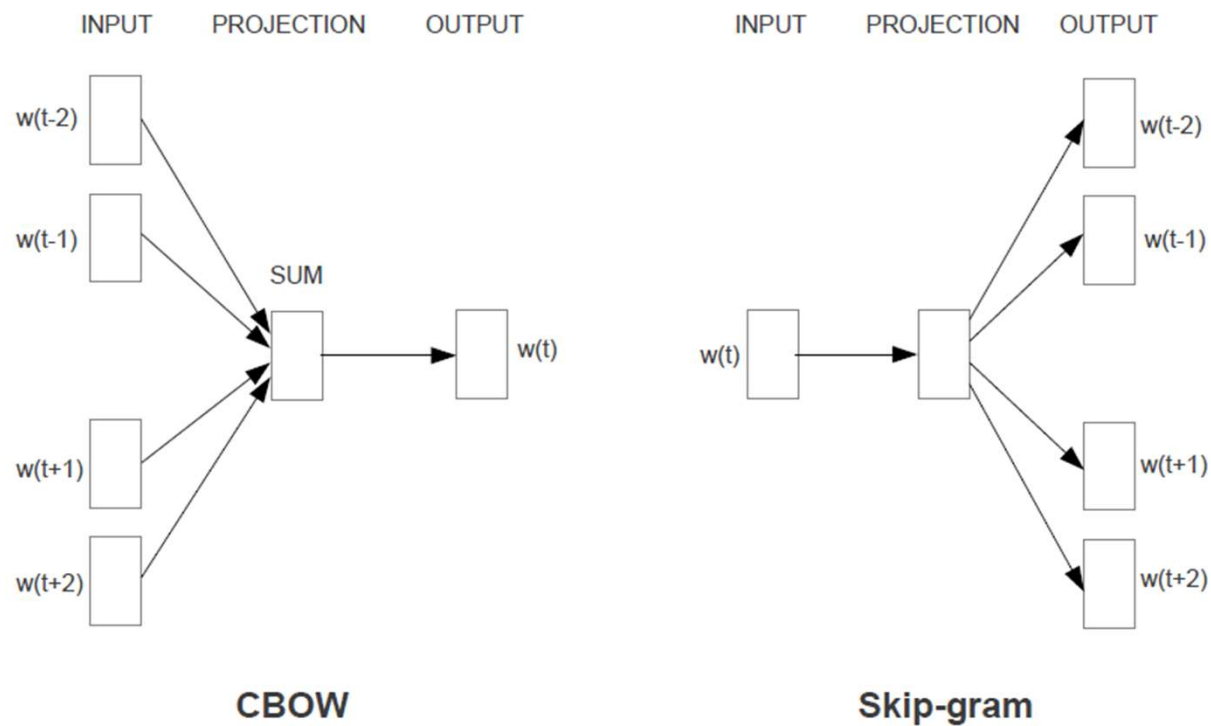
The quick brown fox jumps over the lazy dog.

In practice, this is an impossible task, but remember that the loss only cares *whether words are relatively likely to be close to each other*. Given a large training set, this approach turns out to work very well.

Given the symmetry between target and context word, there are different interpretations whether targets are inferred from context, or context from target. The two pictures are equivalent.

Generating a word embedding

From the original paper:



Source: "Efficient Estimation of Word Representations in Vector Space", Mikolov et al 2013

Generating a word embedding: variations

Hierarchical softmax

The primary time sink when evaluating $P(t_i|c_i, \{W, E\})$ is the softmax layer, in particular the denominator $(\sum_j e^{e_j \cdot w_t})$, which can contain a huge number of terms. A different approach is to replace the softmax function with a tree structure, where the most 'likely' terms are placed at the top. See <https://ruder.io/word-embeddings-softmax/> for a good description.

Sample only close words

The skip-gram can be tweaked to give higher weight to close neighbours by preferentially sampling context words that are close to the target word. This may also cause the model to converge faster.

Negative sampling

Instead of comparing the probabilities across all possible targets (as the softmax does), we can just compare predictions between the correct and a selection of incorrect pairings:

$$P(t_i|c_i, \{W, E\}) = 1$$

$$P(t_j|c_i, \{W, E\}) = 0$$

The incorrect targets are then sampled from all possible targets.

Comparison of CBOW and Skip-gram

CBOW and Skip-gram are both packaged in the program Word2Vec (<https://code.google.com/archive/p/word2vec/>). According to that documentation, the pros and cons are:

The hyper-parameter choice is crucial for performance (both speed and accuracy), however varies for different applications. The main choices to make are:

- *architecture: skip-gram (slower, better for infrequent words) vs CBOW (fast)*
- *the training algorithm: hierarchical softmax (better for infrequent words) vs negative sampling (better for frequent words, better with low dimensional vectors)*
- *sub-sampling of frequent words: can improve both accuracy and speed for large data sets (useful values are in range $1e-3$ to $1e-5$)*
- *dimensionality of the word vectors: usually more is better, but not always*
- *context (window) size: for skip-gram usually around 10, for CBOW around 5*



Quiz:

The two algorithms generate two embedding vectors for each word, e_i & w_i , what could be the advantage of that?

Hint 1: what should be $P(t_i|c_i, \{W, E\})$ if $i = j$ with single-word context?

Hint 2: What does this say about $e_i^T e_i$?

Answer:

A word is rarely its own context. Thus, we would expect a small (0) inner product between the word vector and itself. However, if we used a single vector, that would mean that $e_i \cdot e_i = 0 \Rightarrow e_i = 0$, which is quite boring.





Break?

Yet another embedding:

GloVe: Global Vectors for representation

Let X_{ij} be the number of times words i and j occur close to each other (meaning that we make a symmetric context around word i , and see if word j is in that). A good context size in this case is $n=10$. Given X_{ij} , we can obtain word vectors w_i and $\tilde{w}_i$ by minimizing the following loss:

$$L = \sum_{j,i \in V} f(X_{ij})(w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}))^2$$

Let us try to motivate the two main factors:

$f(X_{ij})$: if X_{ij} is small, we consider it to be somewhat noisy, and do not wish to give it too much weight. In particular, $\log(0) = -\infty$, which we do not want to include in the sum. Hence, we would like an $f(X_{ij})$ which is a growing function of X_{ij} , and where $f(0) = 0$. The inventors suggest

$$f(x) = \begin{cases} \left(\frac{x}{x_{max}}\right)^\alpha & \text{if } x < x_{max} \\ 1 & \text{else} \end{cases}$$

In the original paper, $\alpha = \frac{3}{4}$, $x_{max} = 100$ was used.

GloVe: Global Vectors for representation

$$w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}):$$

Let us pretend that our text corpus is partitioned into a long string of ‘contexts’ (n_c in total), and let x_i be an indicator function for whether word i is in a given context (below are chosen ‘det’ and ‘er’):

Der er et yndigt land, det står	med brede bøge, Nær salten østerstrand	nær salten østerstrand, Det	bugter sig i bakke dal, det hedder	gamle Danmark, Og det er Frejas,	sal, og det er Frejas sal. Der sad	i fordums tid, de harnisk klædte kæmper	Udhvilede fra strid, udhvilede	fra strid, Så drog de frem	til fjenders mén, nu hvile
---------------------------------	--	-----------------------------	------------------------------------	----------------------------------	------------------------------------	---	--------------------------------	----------------------------	----------------------------

‘det’:	x_i :	1	0	1	1	1	0	0	0	0
‘er’:	x_j :	1	0	0	0	1	1	0	0	0

If we naively assume that x_i and x_j are uncorrelated random variables, then:

$$\frac{X_{ij}}{n_c} = E[x_i x_j] = E[x_i]E[x_j] \Rightarrow \log(X_{ij}) - \log(n_c) = \log(E[x_i]) + \log(E[x_j])$$

Let us simplify by letting $b_i = \log(E[x_i])$, $b_j = \log(E[x_j])$, and ignoring $\log(n_c)$ since it does not depend on θ and will not influence L :

$$\log(X_{ij}) = b_i + b_j$$

GloVe: Global Vectors for representation

This means that under the assumption that words i and j appear independently of each other (x_j and x_i are uncorrelated),

$$b_i + b_j - \log(X_{ij}) = 0$$

However, this is a flawed assumption, and we model the error by a linear term depending on both i and j :

$$w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}) = 0$$

'error-term'

We estimate the error term (as well as the b-terms) by minimizing the square of the whole expression:

$$\min \left(w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}) \right)^2$$

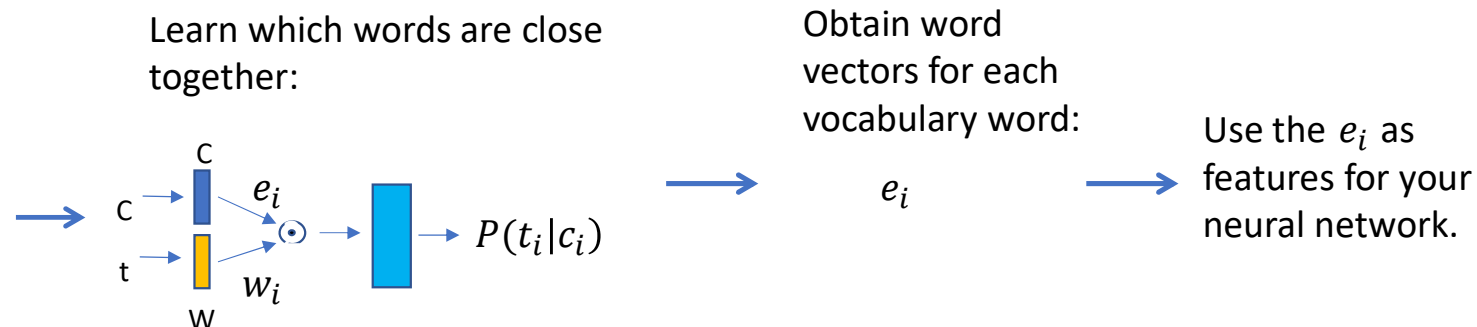
All together:

$$L = \sum_{j,i \in V} f(X_{ij}) (w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}))^2$$

Finally, I should stress that the original paper arrives here through a much more elaborate derivation. But I think this is a useful way of thinking about it.

Embedding recap

Lots of relevant text
(unlabeled):



Embeddings are a way to use unsupervised training to improve performance of a supervised network.

Visualization intermezzo: t-SNE

't-distributed Stochastic Neighbor Embedding' is a handy way of visualizing nD information in 2D (or 3D), which excels in keeping clusters together. It was proposed by Hinton in two papers from 2003 and 2008. So as plots go, it's a pretty new invention.

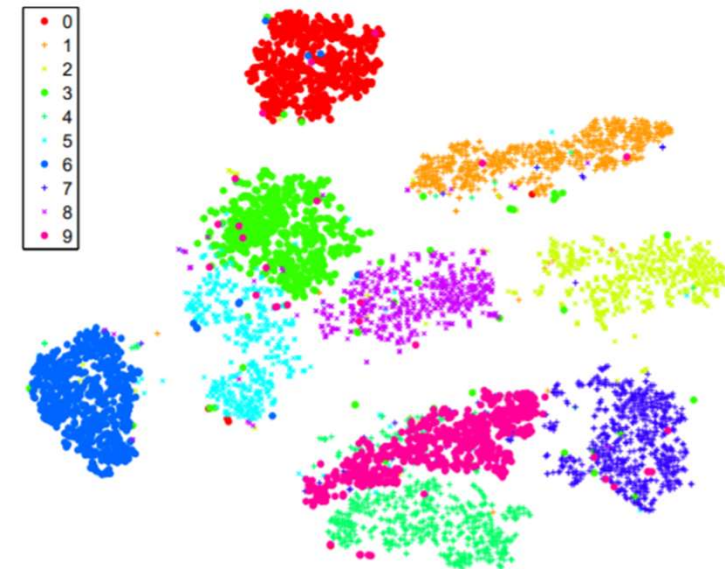
While it does not utilize deep learning, it is often used in visualizing data and activation maps (and it's invented by Hinton), so it is quite popular in this field.

It works by comparing conditional distributions of neighbourhoods in nD and 2D (3D):

$$p_{j|i} = \frac{d_n(i, j)}{\sum_{k \neq i} d_n(i, k)} \quad q_{l|m} = \frac{d_2(l, m)}{\sum_{o \neq i} d_2(l, o)}$$

$p_{j|i}$ is a 'probability' of point j being a neighbour of point i , $d_n(i, j)$ is a similarity measure of points i and j in nD (and similar for $q_{j|i}$ and $d_2(i, j)$).

MNIST data plotted with t-SNE:



Visualization intermezzo: t-SNE

$$p_{j|i} = \frac{d_n(i, j)}{\sum_{k \neq i} d_n(i, k)} \quad q_{i|m} = \frac{d_2(i, j)}{\sum_{o \neq i} d_2(i, k)}$$

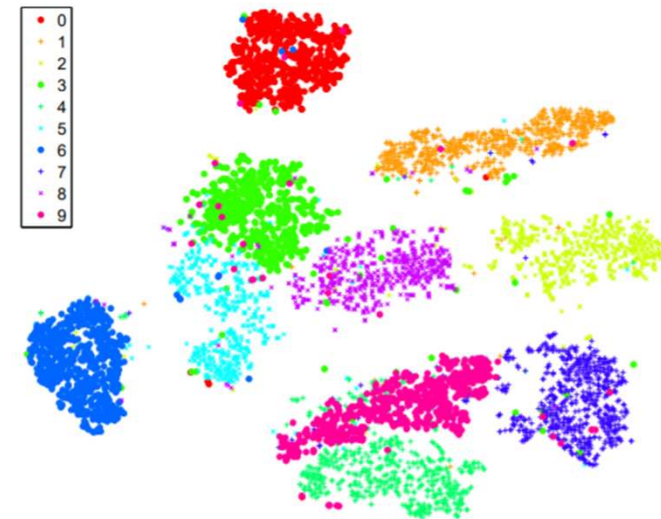
In the standard implementation, $d_n(i, j) = \exp(-|x_i - x_j|^2 / \sigma_i)$ and $d_2(i, j) = (1 + |y_i - y_j|^2)^{-1}$ (t-distribution with 1 degree of freedom)

We can now choose y_i as the 'projection' of x_i in the low-dimensional space, such that the Kulback-Leibler divergence between $p_{j|i}$ and $q_{j|i}$ is minimized (using gradient descent):

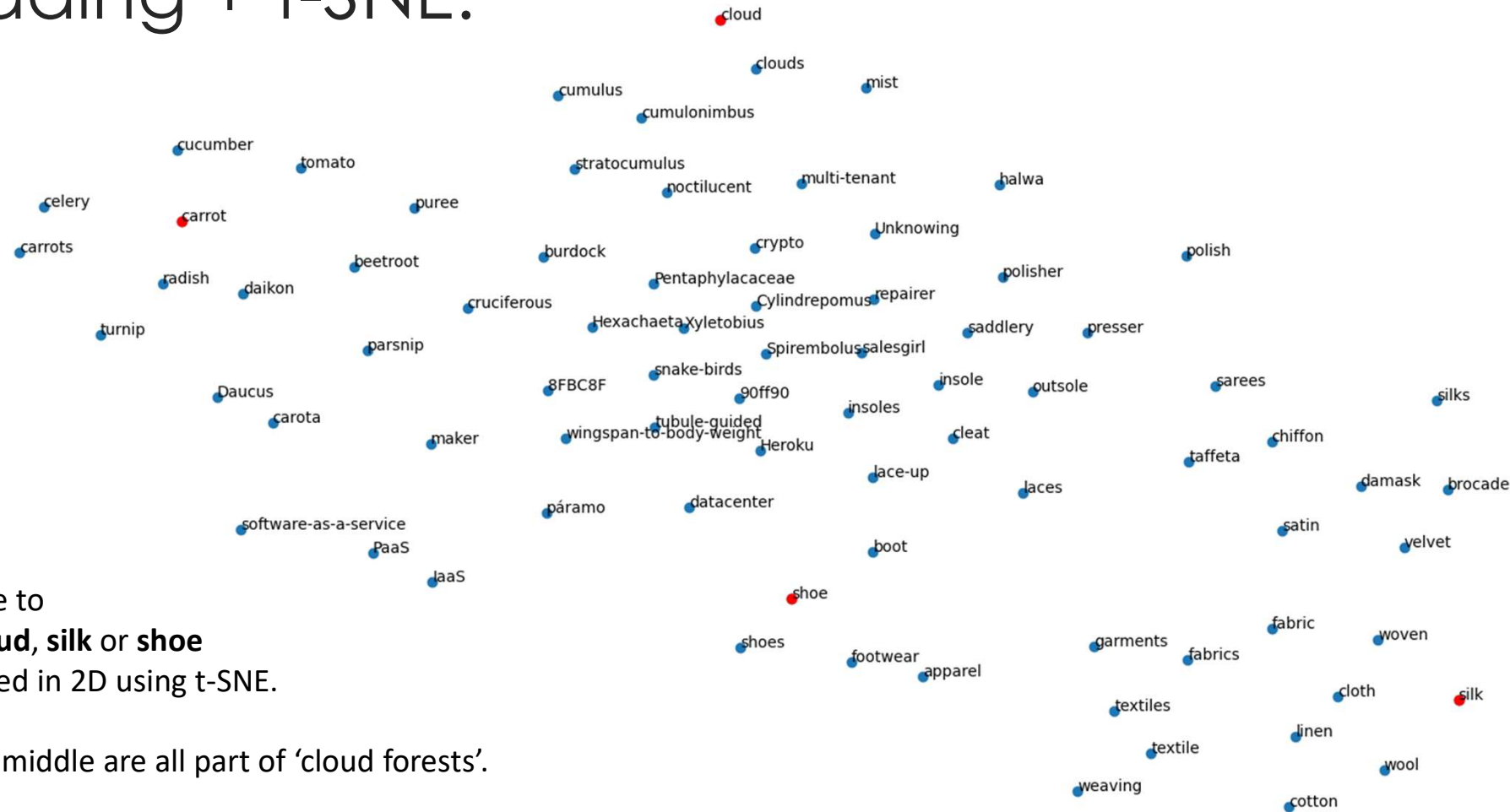
$$L = \sum_{i,j} p_{i|j} \log \frac{p_{i|j}}{q_{i|j}}$$

Finally, σ_i are chosen such that each x_i has roughly the same number of neighbours (and the target number is a free parameter, called the 'perplexity').

MNIST data plotted with t-SNE:



Embedding + t-SNE:



Here, words close to either **carrot**, **cloud**, **silk** or **shoe** have been grouped in 2D using t-SNE.

The plants in the middle are all part of 'cloud forests'.



...and another thing :

It turns out that since we are projecting words into a feature space, where the dimensions are loosely representing concepts, we can actually do arithmetic with words:

"Man relates to woman as king relates to queen" ->

"man" - "woman" = "king" - "queen" => "man" + "queen" - "woman" = "king"



In reality, the embedding vector for "king" will only be really close to this, but if you print the closest five vectors to a given point, you will often find the word you're looking for.

You will play around with this in the exercises 😊





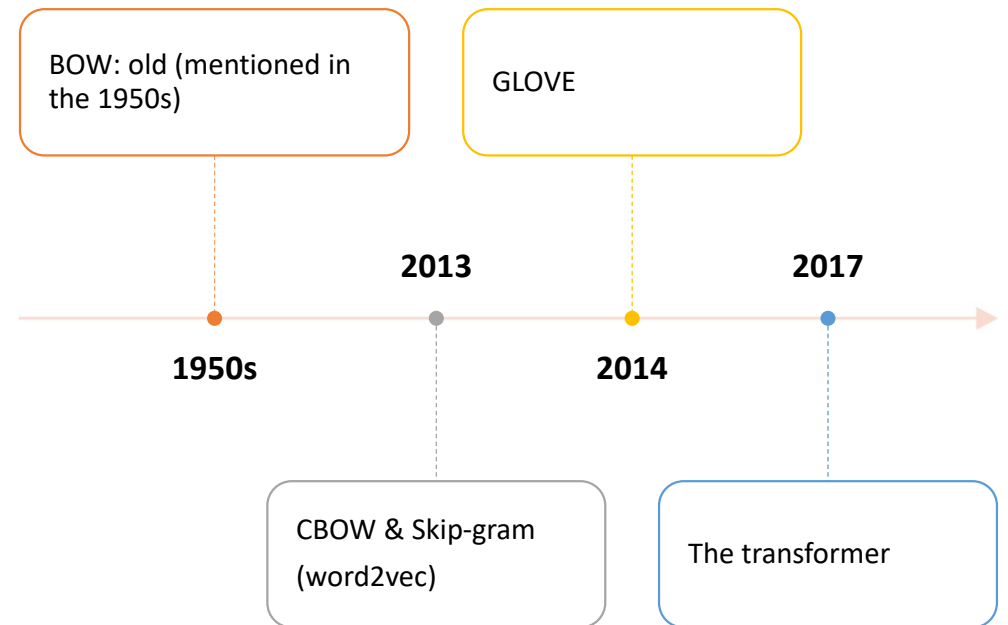
Discussion:

What is it about embeddings that allows this sort of word arithmetic?

Break 😊



Timeline:

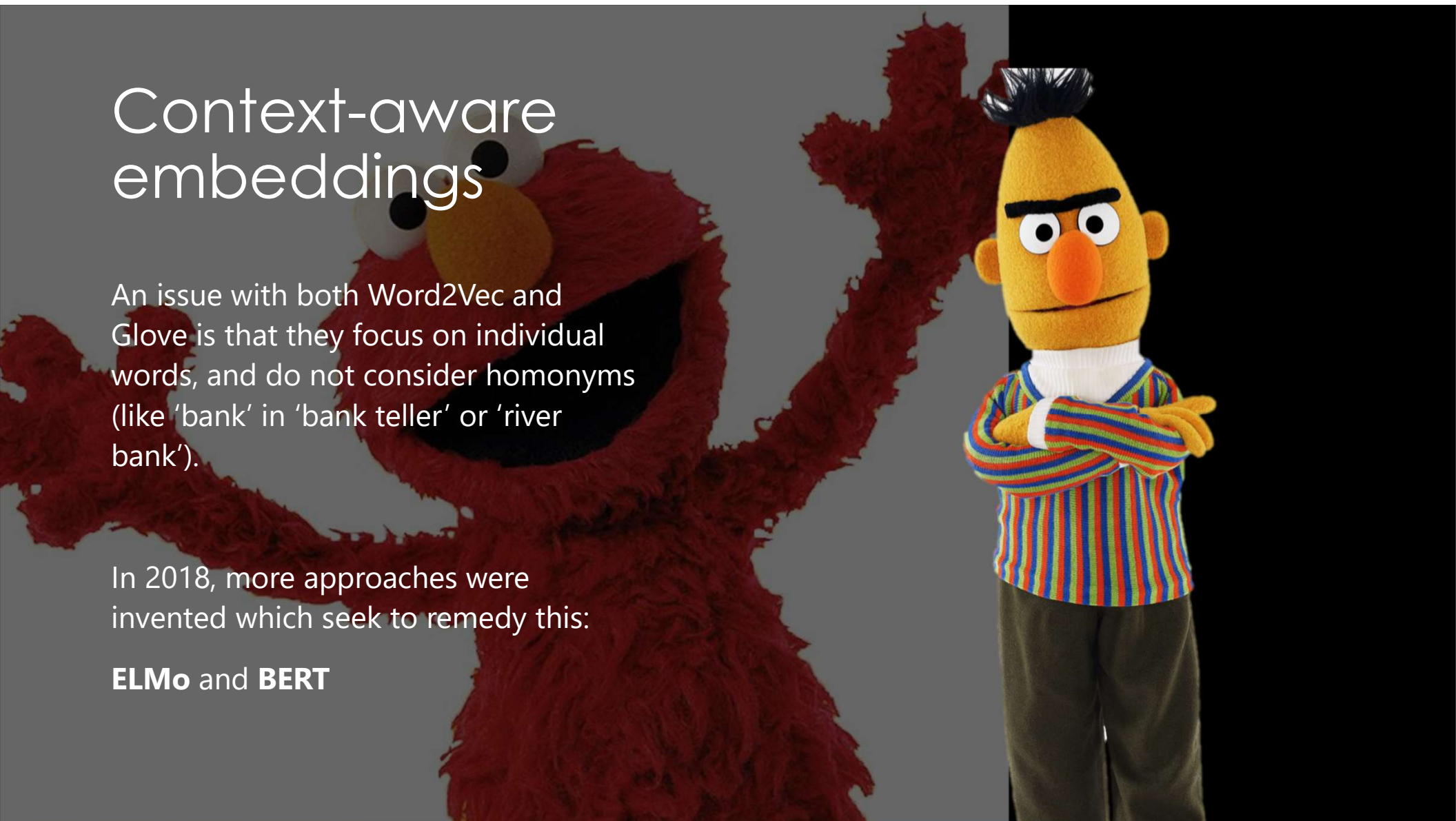


Context-aware embeddings

An issue with both Word2Vec and GloVe is that they focus on individual words, and do not consider homonyms (like 'bank' in 'bank teller' or 'river bank').

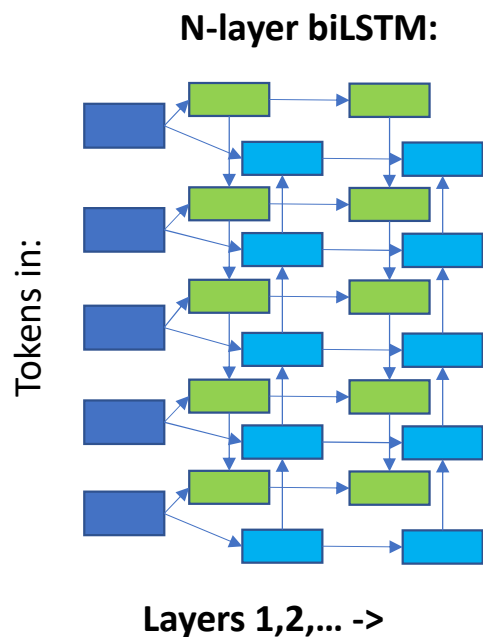
In 2018, more approaches were invented which seek to remedy this:

ELMo and **BERT**



ELMo : Embeddings from Language Models

ELMo is a considerably more involved approach to generating language representations.
Consider a multi-layer biLSTM network:



We can train this in the same auto-encoder manner as with Word2Vec or GloVe, by maximizing the conditional probability:

$$L(\theta) = - \prod_i^N P(t_i | \{t_j\}_{j < i})$$

After we have trained our network, we can use the activations in each layer as embedding vectors. This means that for an L-layered biLSTM, we get $2L+1$ representation vectors for each input token.

These vectors (or a linear combination) can be used instead of embedding vectors.

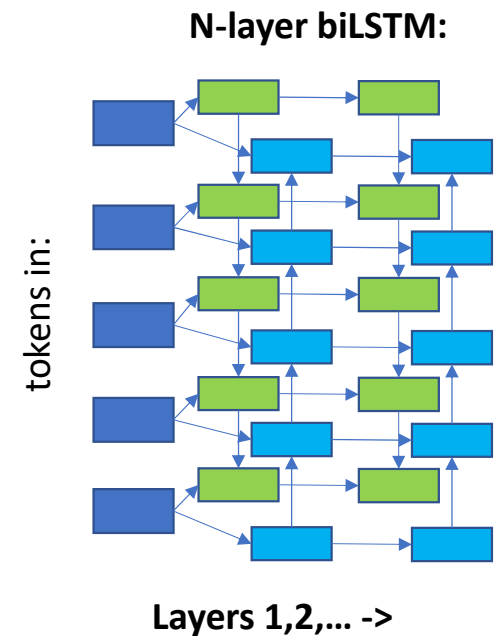


ELMo : Embeddings from Language Models

A note on the ELMo input:

Whereas Word2Vec and GloVe worked with whole words, ELMo works on character sequences. This is done by converting each character to an integer, and running a CNN on the series of integers. This output is described as 'tokens'.

Since the input is still 'raw' text, it makes little difference for the user, but this approach means that the network is much better at handling words with multiple spellings, or where related words have related spellings (color, colour, colorful, for example).



Tokenization

ELMo is the beginning of 'modern' NLP, and its input was *tokens*.

A token is something in-between a letter and a word. For instance, the 'Meta LLAMA tokenizer' does something like this:



<https://lunary.ai/llama3-tokenizer>

<https://arxiv.org/pdf/2302.13971>

<https://github.com/google/sentencepiece>

Tokenization

As an example, the LLAMA tokenizer is based on 'byte-pair-encoding', which is essentially a compression algorithm:

" We iteratively count all symbol pairs and replace each occurrence of the most frequent pair ('A', 'B') with a new symbol 'AB'. Each merge operation produces a new symbol which represents a character n-gram. Frequent character n-grams (or whole words) are eventually merged into a single symbol, thus BPE requires no shortlist."

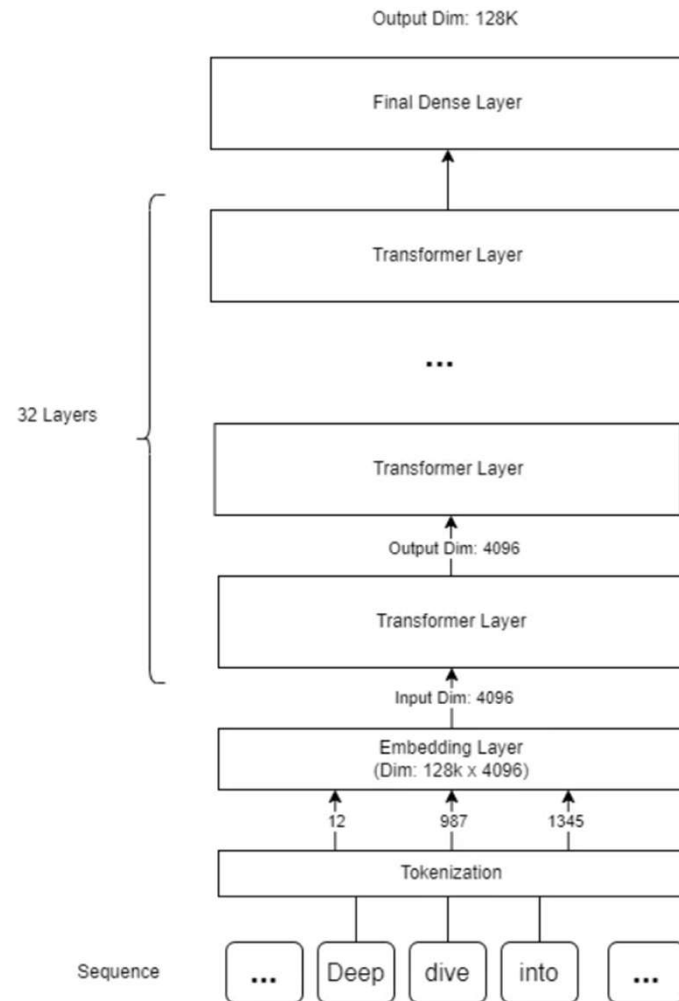
Using the token vocabulary, a one-hot-encoded list of tokens ('o') can then be multiplied with an embedding matrix, E:

$$v = E\mathbf{o}$$

Usually, E is a (quite large) model parameter.

Llama example

Here is seen a pretty good description from a 'Llama'-blog.





BERT: Bidirectional Encoder Representations for Transformers

Where ELMo build on layered biLSTM networks, BERT used the newer transformer architecture. In this case we are only using the 'encoder' part of the transformer, but it is quite large: up to 24 layers, hidden size 1024 (d_{model}) and 16 attention-heads. This results in 340 million parameters...

As with all other approaches we have discussed, the trick is to train BERT on a large 'unlabeled' dataset.

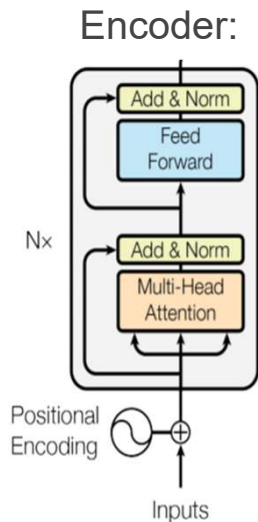
In this case, BERT is trained on two tasks simultaneously:

- guessing a missing ('masked') word
- Determining whether two sentences are neighbours or not

'Masking' here means that BERT is given the entire sentence as context, but with 15% of words having been exchanged with a 'mask' token.

The sentence classification is done by always inputting two sentences, with a delimiter between.

Like ELMo, BERT does not use word-level embeddings, nor one-hot-encoded vectors, but instead tokens.



BERT: Bidirectional Encoder Representations for Transformers



Finetuning:

Where ELMo (and Word2Vec & GloVe) supply feature vectors to a separately developed model, BERT is the model. The idea is that any task which concerns either classification of a given sentence, or the relationship between two sentences, are so close to what BERT has been trained on, that we simply add an additional output layer, and use regular finetuning (of the entire network) to get accurate outputs. From the paper:

"Compared to pre-training, fine-tuning is relatively inexpensive. All of the results in the paper can be replicated in at most 1 hour on a single Cloud TPU, or a few hours on a GPU, starting from the exact same pre-trained model."

Please note that BERT is not a language generator.

While it is possible to force BERT to generate text, it is not an easy or good approach.

ERNIE: Enhanced Language Representation with Informative Entities

And yes, there's also been an ERNIE model, developed by Baidu, but it didn't receive quite as much attention in the west (the git repository is in chinese...).



Source: <https://arxiv.org/abs/1907.12412>



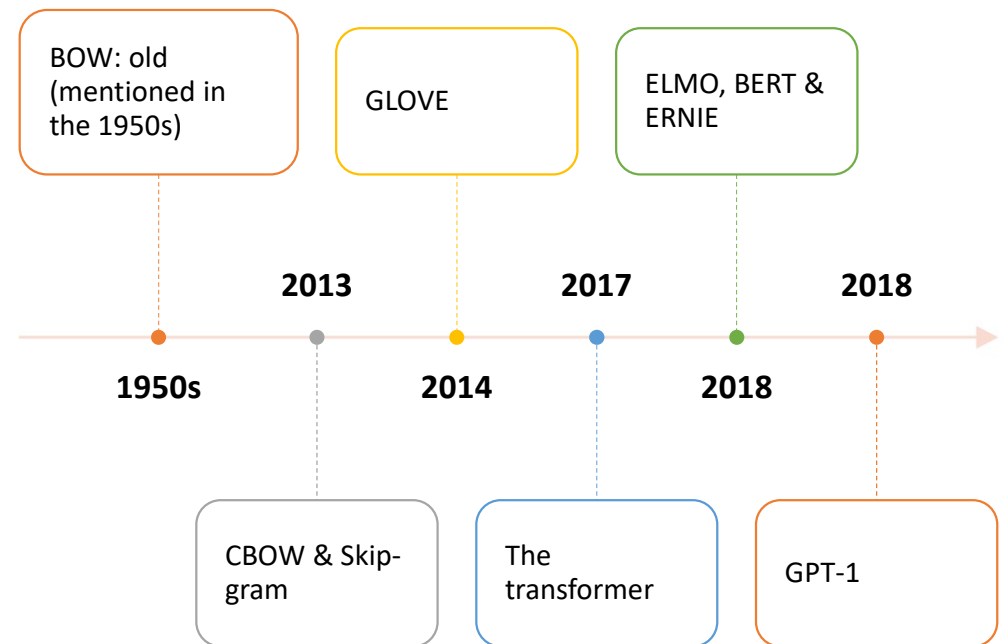
And then
things took
off...

BERT gave rise to a number of clones:

- ❖ BioBERT
- ❖ DistilBERT
- ❖ RoBERTA

Also, this was the beginning of the transformer revolution, which we'll talk more about now.

Timeline:



GPT-1

In 2018, OpenAI presented their 'Generative Predictive Transformer' (GPT). GPT was pretrained to maximize the by-now well known

$$\sum_i \log P(t_i | t_{i-k}, t_{i-k+1}, \dots, t_{i-1})$$

In other words, we are predicting a token based on the previous k tokens.

They then finetuned that for various tasks:

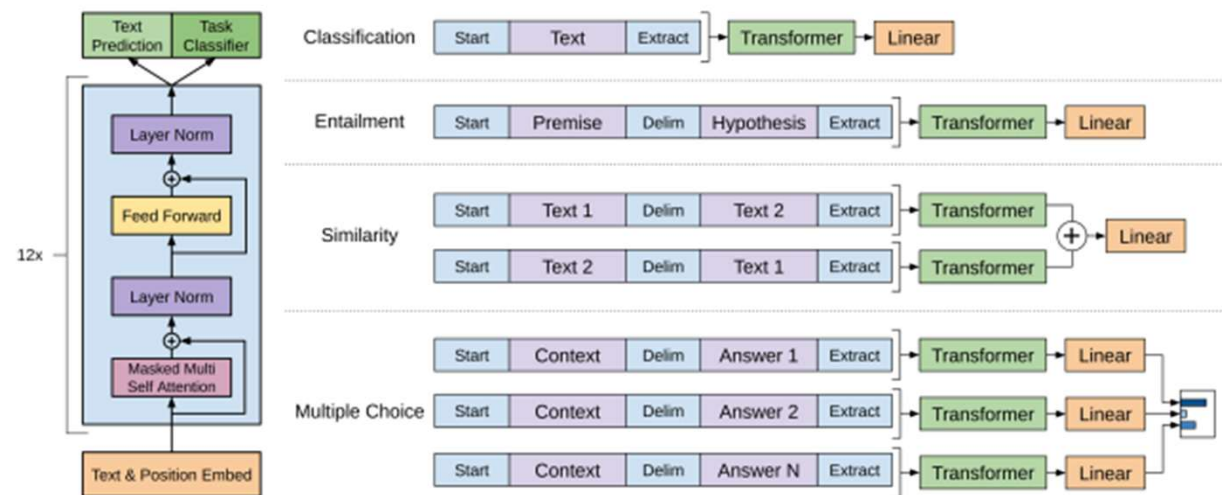
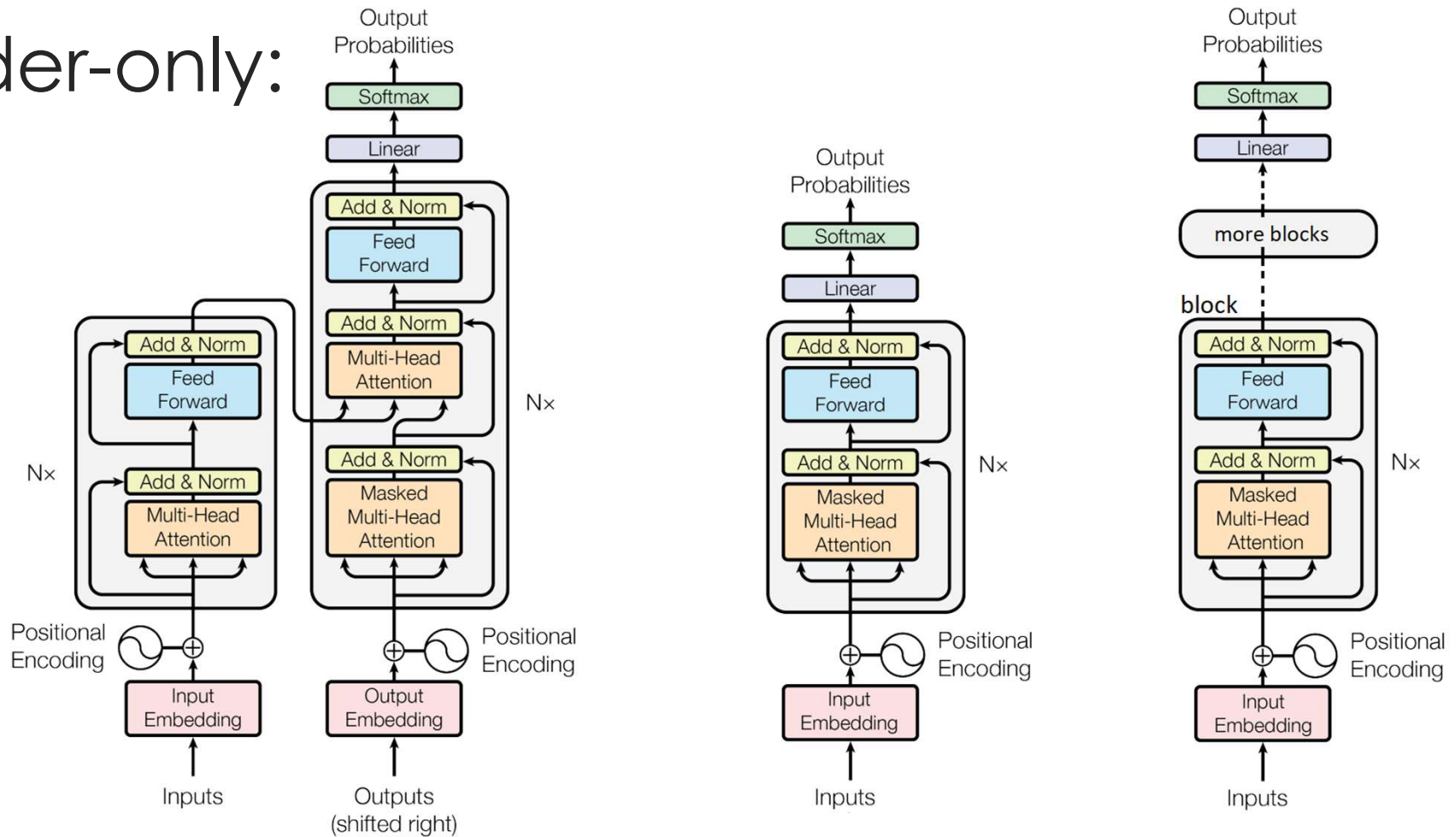


Figure 1: **(left)** Transformer architecture and training objectives used in this work. **(right)** Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

Decoder-only:



GPT-2: 'the Model that's too dangerous to release'

In 2019, OpenAI presented GPT-2. It was largely an extension of GPT-1, with a larger context size ($k=512$) and some more normalization.

They found that for a large selection of different text corpora, GPT-2 outperforms most previous attempts at guessing missing words, without any finetuning at all.

GPT-2 was shown to be able to write poetry, and could output text which was often mistaken for human generated.

Upon release, OpenAI wrote a blog post discussing the implications of these new language models:
<https://openai.com/blog/gpt-2-1-5b-release/>

https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

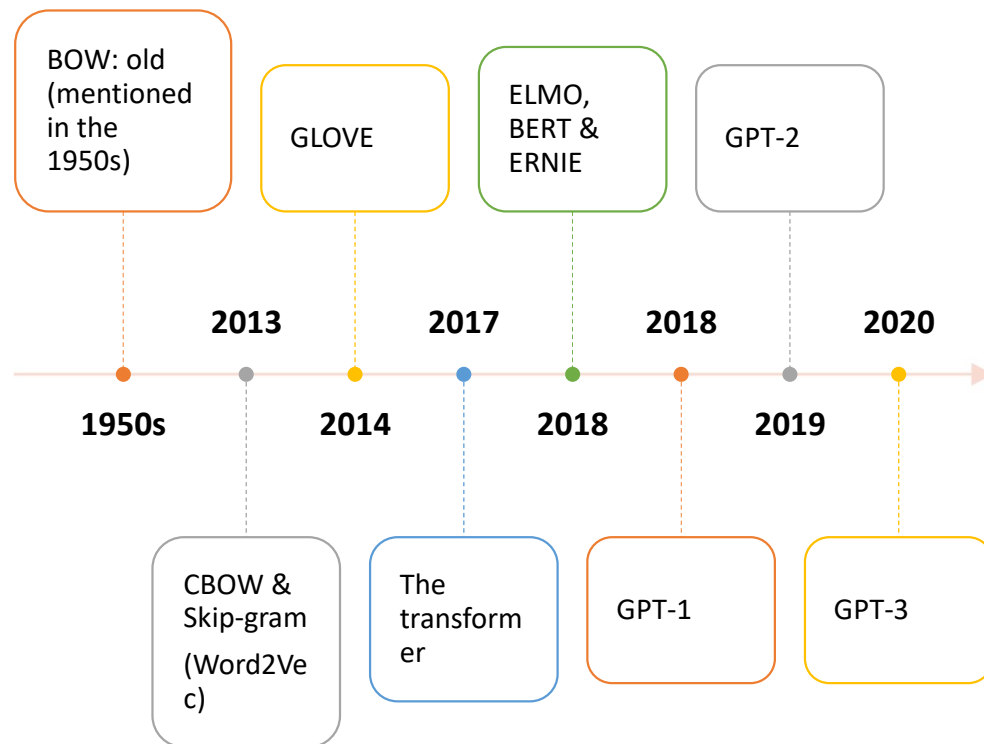
GPT-3

Then, in 2020, OpenAI presented their 3rd GPT iteration. The architecture was the same as for GPT-2, just bigger (with a slightly more involved implementation of self attention). Context size was now 2048 tokens. Training data was also bigger.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

Timeline:



GPT-3

GPT-3 can generate text, with even a short prompt:

Write an original poem from scratch, using the style of Shakespeare's sonnets

When I am old and grey and full of sleep,
And nodding by the fire, take down this book,
To a young man I loved, and see if I
Was really a greatful lover, or a thief.

Bare with me as I try to make amends
And admit the faults that I have done
For I was young and unwise and full of lust
And I took what I could from who I wanted

For I was given a gift that I could not use
But I'm not bitter, because I see now
That I was given a second chance
For all the things I could not do

Now, I love with all my heart
And I would give anything to keep him safe
For I know that I now have the key
To a second chance and a new life

<https://techcommunity.microsoft.com/t5/educator-developer-blog/gpt3-openai-3-demos-that-will-let-you-rethink-about-ai/ba-p/3521075>

Or perform translation:

Translate the text below in Italian:

When I am old and gray and full of sleep,
And nodding by the fire, take down this book,
To a young man I loved, and see if I
Was really a greatful lover, or a thief.

Quando sarò vecchio e grigio e pieno di sonno,
e annuendo davanti al fuoco, prendi questo libro,
per un giovane che ho amato, e vedi se
ero davvero un amante grato o un ladro.]



These are examples of 'few-shot learning', which we have discussed before:

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 cheese =>                   ← prompt
   .....
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 sea otter => loutre de mer    ← example
3 cheese =>                    ← prompt
   .....
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French:  ← task description
2 sea otter => loutre de mer    ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese =>                    ← prompt
   .....
```

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



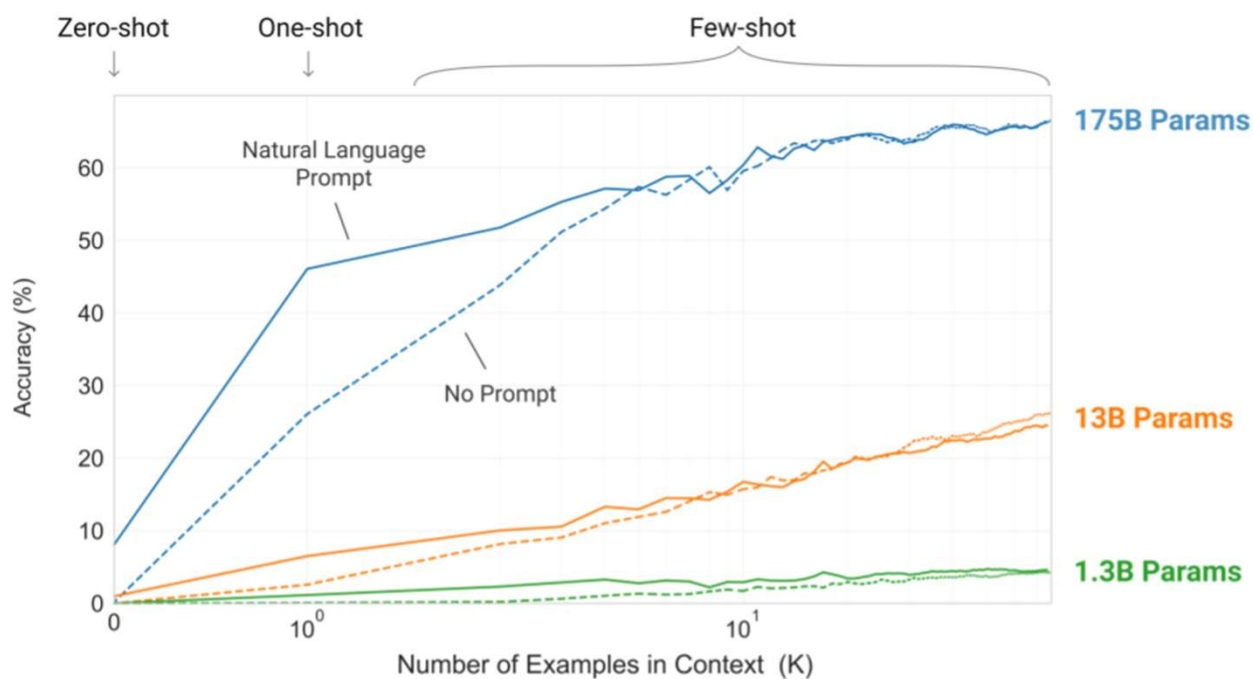
Source: <https://arxiv.org/pdf/2005.14165.pdf>

This is a benefit of huge models:

One of the most exciting things to come out of the large NLP models is 'few shot learning' or 'meta learning'.

In its ideal form, the network is simply given a context of similar sentences, and then it can solve similar NLP tasks without any actual training taking place.

To the right is shown accuracy of various GPT-3 versions, using only 'meta learning'.



How are GPT-1/2/3 trained?

The GPT models are trained to finish text, possibly very long text samples. The input context is 1024 and 2048 'tokens' (a mix between single letters and common words).

As was discussed last week, the model simply gets the attention weights for the entire input context, and is tasked with continuing the text.

To the left is an example of the test data used in evaluating GPT-3:

Context → Helsinki is the capital and largest city of Finland. It is in the region of Uusimaa, in southern Finland, on the shore of the Gulf of Finland. Helsinki has a population of , an urban population of , and a metropolitan population of over 1.4 million, making it the most populous municipality and urban area in Finland. Helsinki is some north of Tallinn, Estonia, east of Stockholm, Sweden, and west of Saint Petersburg, Russia. Helsinki has close historical connections with these three cities.

The Helsinki metropolitan area includes the urban core of Helsinki, Espoo, Vantaa, Kauniainen, and surrounding commuter towns. It is the world's northernmost metro area of over one million people, and the city is the northernmost capital of an EU member state. The Helsinki metropolitan area is the third largest metropolitan area in the Nordic countries after Stockholm and Copenhagen, and the City of Helsinki is the third largest after Stockholm and Oslo. Helsinki is Finland's major political, educational, financial, cultural, and research center as well as one of northern Europe's major cities. Approximately 75% of foreign companies that operate in Finland have settled in the Helsinki region. The nearby municipality of Vantaa is the location of Helsinki Airport, with frequent service to various destinations in Europe and Asia.

Q: what is the most populous municipality in Finland?

A: Helsinki

Q: how many people live there?

A: 1.4 million in the metropolitan area

Q: what percent of the foreign companies that operate in Finland are in Helsinki?

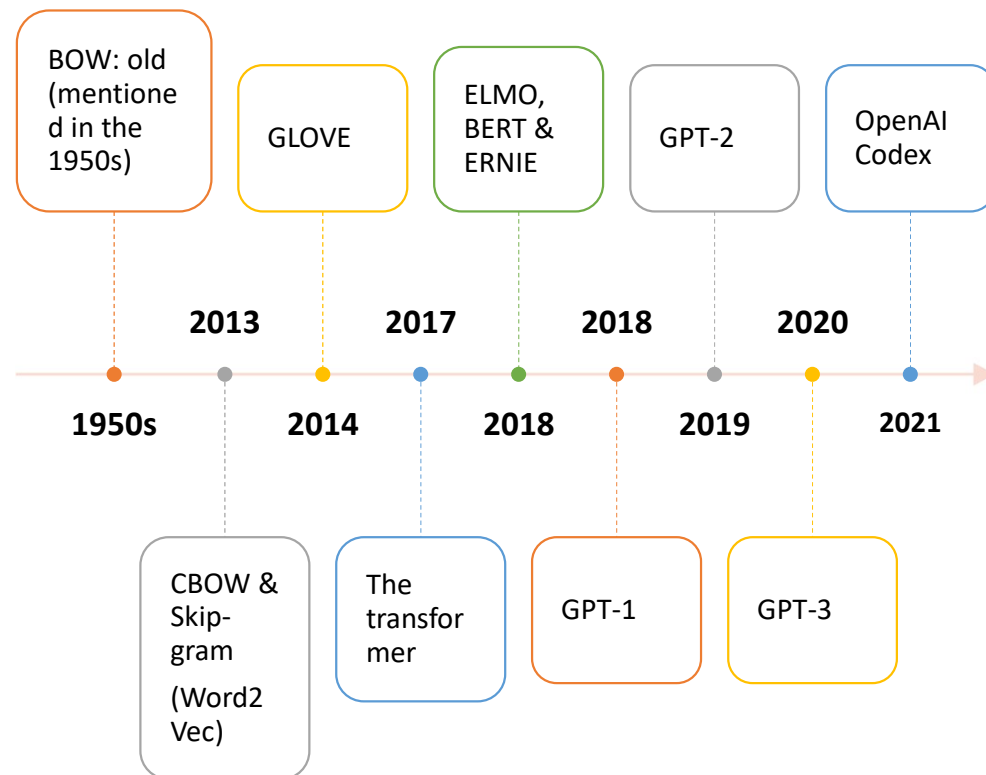
A: 75%

Q: what towns are a part of the metropolitan area?

A:

Target Completion → Helsinki, Espoo, Vantaa, Kauniainen, and surrounding commuter towns

Timeline:



OpenAI Codex

In 2021, OpenAI presented Codex, which was a version of GPT-3 trained on Github repositories. Given a real language input, Codex can output working code in over a dozen languages (python is the best).

Microsoft and OpenAI since implemented Codex as the 'Github Copilot' product, which can help programmers in real time, generating code based on comments from the programmer.

Instruct-GPT: 2022

InstructGPT was trained to mimic human conversation.

It was based on GPT-3, but 'fine tuned' on new text exchanges between humans, and received feedback on which responses humans found most satisfactory in a text conversation.

We call this latter step '**Reinforcement Learning from Human Feedback**', and it essentially amounts to teaching the model how to navigate a conversation, just like we might teach a robot to navigate a maze or play a game.

This new data consisted of about 120 000 conversations (tiny compared to the 45TB used to train GPT-3).



ChatGPT: 2022

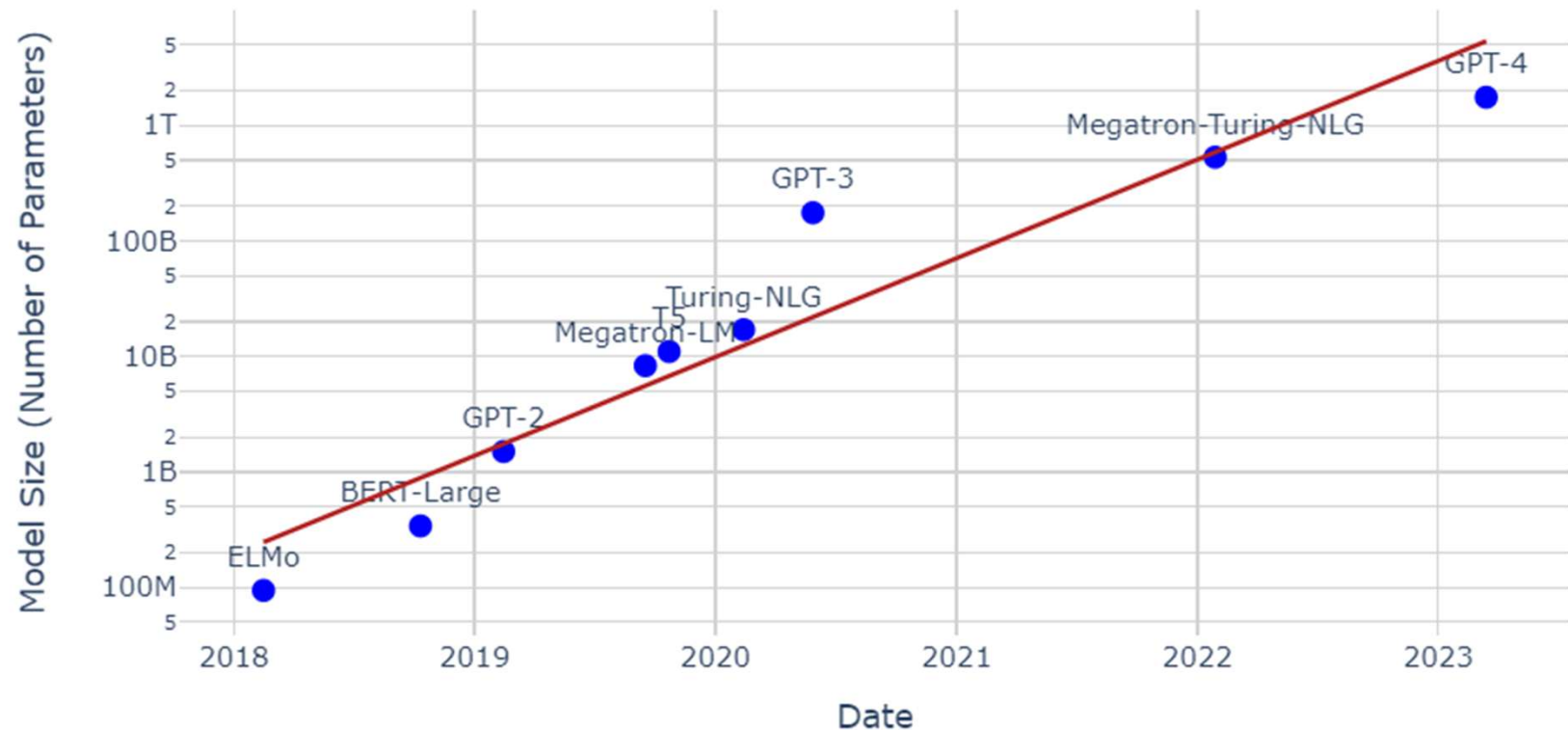
The first chatGPT was essentially instruct-GPT.

The developers simply added slightly more elaborate conversation data, and taught to the model to be more critical of the input.

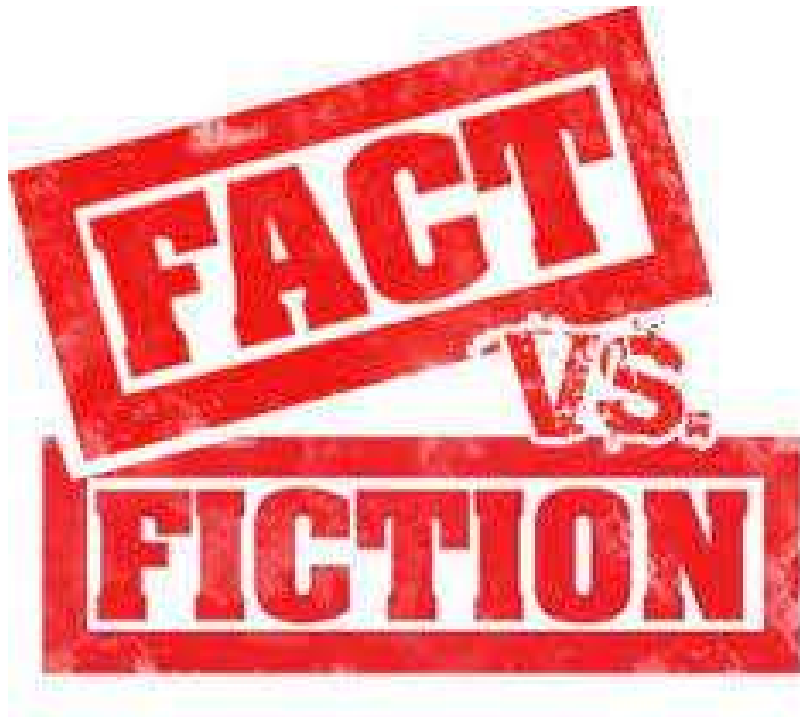


"Create an image of a friendly, helpful AI assistant. It has a blue body and a screen that displays text conversations. It has a confident and approachable expression and is surrounded by gears, circuit boards, and other technology-related items to suggest that it is a highly advanced digital being."

Across these approaches, we see an exponential increase in model size:



Note: GPT-4 is based on rumours.



Question:

How can generative models such as the GPT family distinguish facts from fiction?



Hallucination

From copilot:

A **chatbot hallucination** refers to a situation where an AI model (like a chatbot) generates information that sounds plausible but is **factually incorrect, irrelevant, or completely fabricated**. This happens because large language models predict text based on patterns in their training data rather than verifying facts in real time.



(copilot continued):

Why they can't be fully eliminated

Probabilistic nature of language models

These models predict text based on patterns, not truth. They don't inherently "know" facts—they approximate them.

Training data limitations

If the data is incomplete, outdated, or biased, the model may fill gaps with plausible but incorrect information.

Open-ended prompts

When asked ambiguous or speculative questions, the model often generates creative answers, which can lead to hallucinations.

What can reduce them significantly

Retrieval-Augmented Generation (RAG): *Combining the model with real-time search or verified databases.*

Fact-checking layers: *Post-processing answers against trusted sources.*

Domain-specific fine-tuning: *Training on curated, high-quality datasets for specialized tasks.*

Even with these techniques, hallucinations can only be minimized—not fully eradicated—because language generation is inherently probabilistic.



Technically, it should be
'*confabulation*':

Hallucination:

... a perception in the absence of an external context stimulus that has the compelling sense of reality.

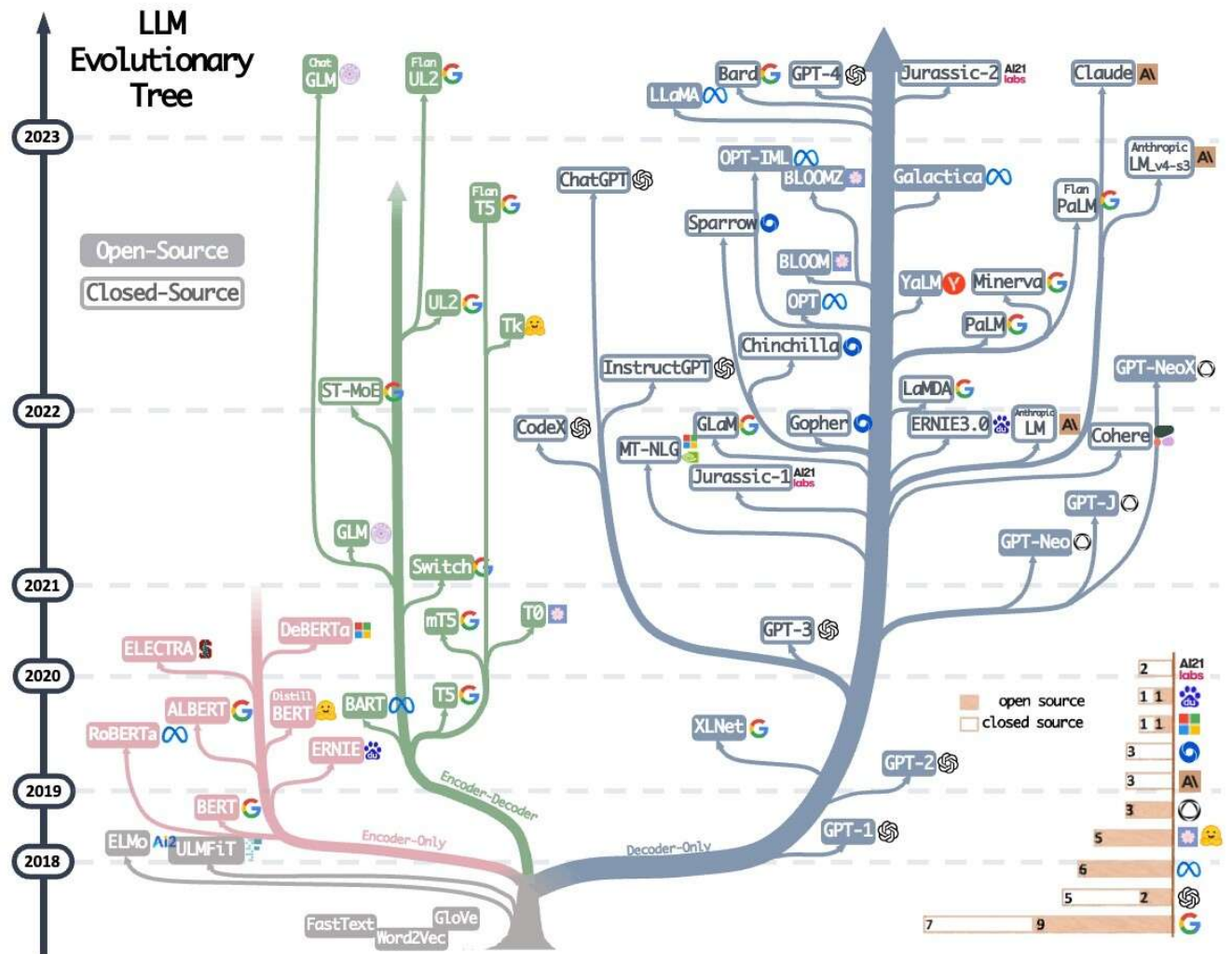
Confabulation:

Confabulation is a memory error consisting of the production of fabricated, distorted, or misinterpreted memories about oneself or the world.

Source: Wikipedia



And if it's not
been said yet,
there's a lot of
models out
there...



Cost of huge models:

- A single training run of BERT is estimated to have emitted 1400 kg of CO₂. That's more than the average emissions from one person taking a transatlantic flight.
- A single training run of GPT-3 took thousands of 'peta-flop days'. Depending on electricity prices & hardware, that is several millions of USD (just in electricity). The developers behind it admitted that they found a small error after having finished training, but it was too costly to retrain the model.
- Modern models are huge investments, in manpower and hardware. Even finished state-of-the-art models are too large to keep on a 'normal' computer. This concentrates power and knowledge within only the largest technology companies.

Sources:

<https://www.technologyreview.com/2019/06/06/239031/training-a-single-ai-model-can-emit-as-much-carbon-as-five-cars-in-their-lifetimes/>

<https://arxiv.org/pdf/2005.14165.pdf>

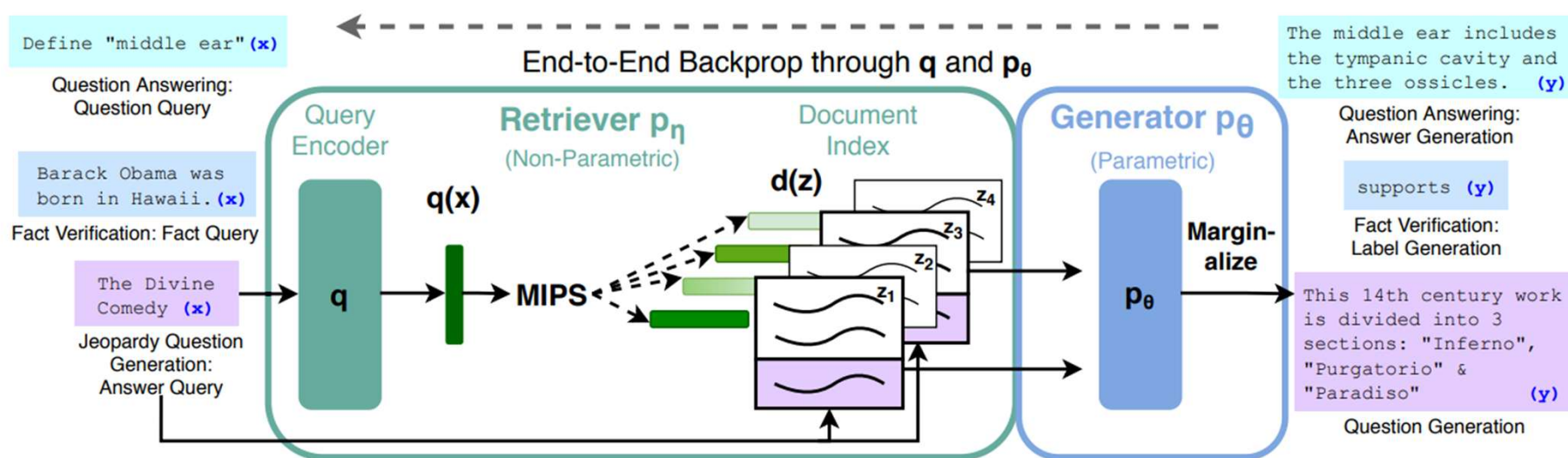
"Modern'er" NLP

(chat-GPT suggests 'hyper modern' or 'cutting-edge')

RAG: Retrieval augmented generation

A cool way to 'finetune' an LLM to work in a specific field is RAG.

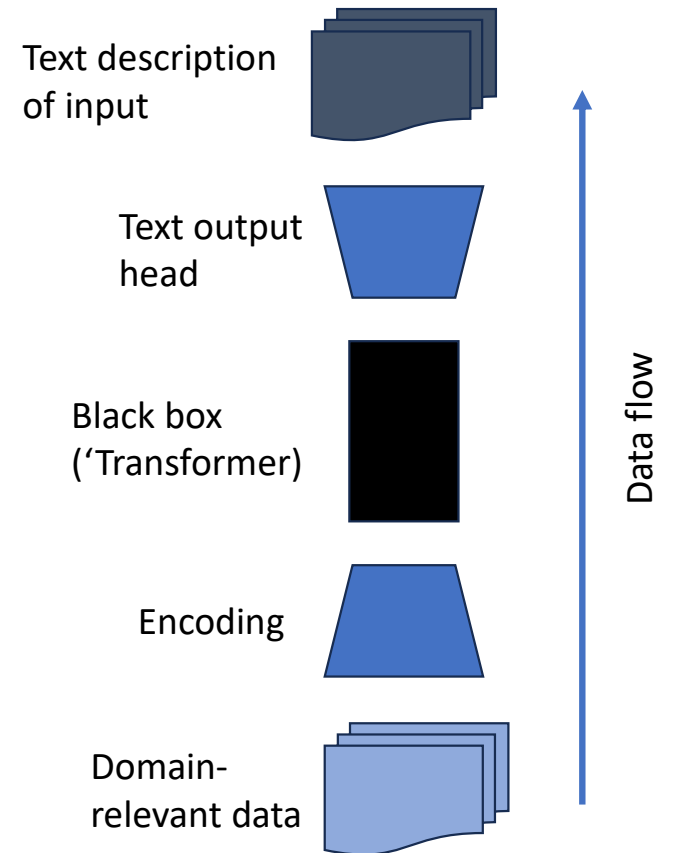
Here, the model is supplied with a database with a bunch of documents (like wikipedia). Each document is represented by an embedding, and prior to generating the output, the model creates an extended context using the most relevant documents.



Foundation models

I have met multiple people wanting to build 'foundation models' for non-language problems, but still using language.

As I understand it, the idea is to present a model with very many different tasks, using language data to mine the 'labels'



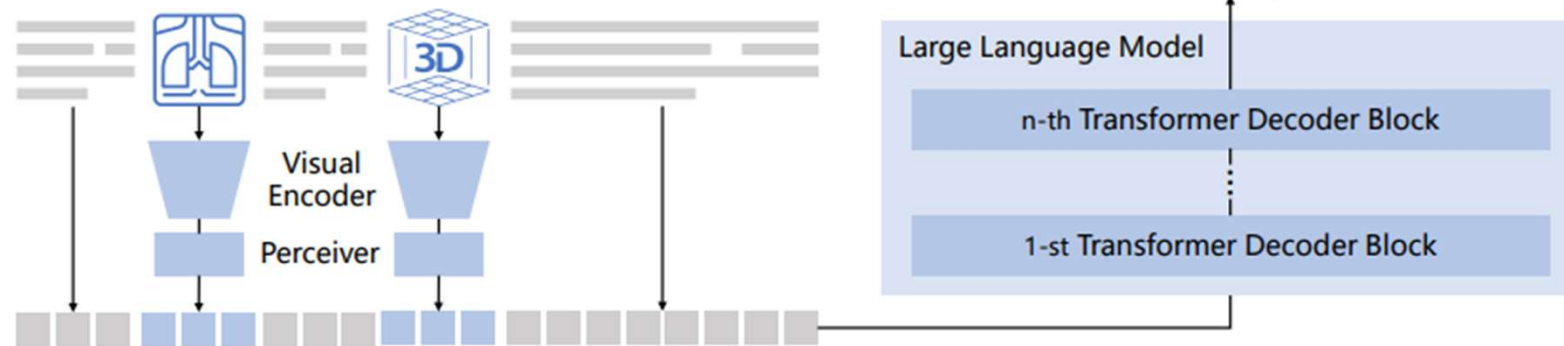
Foundation models

One of the more serious approaches I have seen is:

[\[2308.02463\] Towards Generalist Foundation Model for Radiology by Leveraging Web-scale 2D&3D Medical Data](#)

c. Architecture Detail

Input: Visual scans (3D or 2D) interleaving with texts



Frameworks

- Langchain
- n8n ('Nathan')
- LlamaIndex

These are frameworks for how to use different large language models as part of other coding projects.



Break?!?





DIY NLP

Getting your hands dirty

3 different approaches:



MAKING YOUR OWN
NETWORK

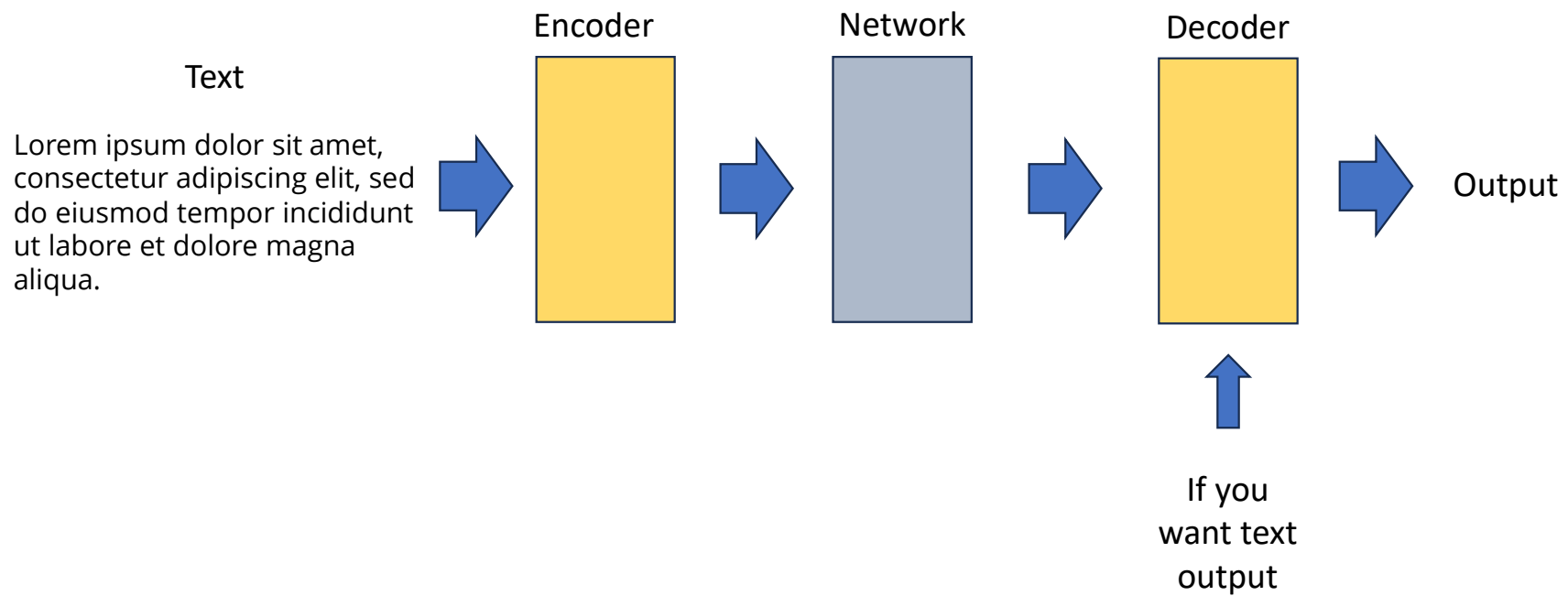


USE A PRETRAINED
TRANSFORMER
(OR MAMBA?)



PROMPT ENGINEERING

Making your own:



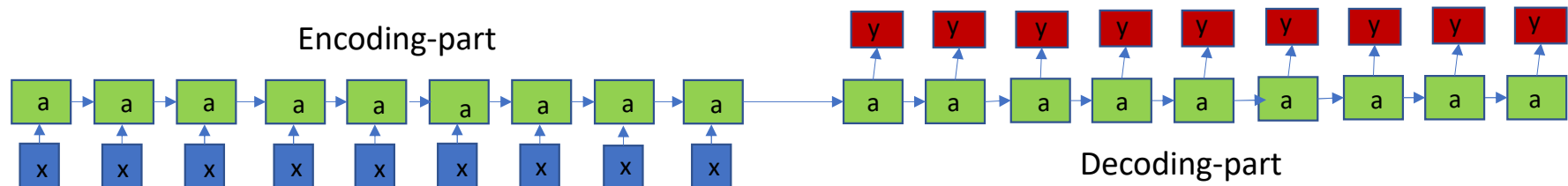
Making your own:

If you have any task that inputs text and outputs a scalar value (class, true/false, marking a specific input word):

Finetune a pretrained model (BERT, GPT-2 or others) with what labeled data you have.

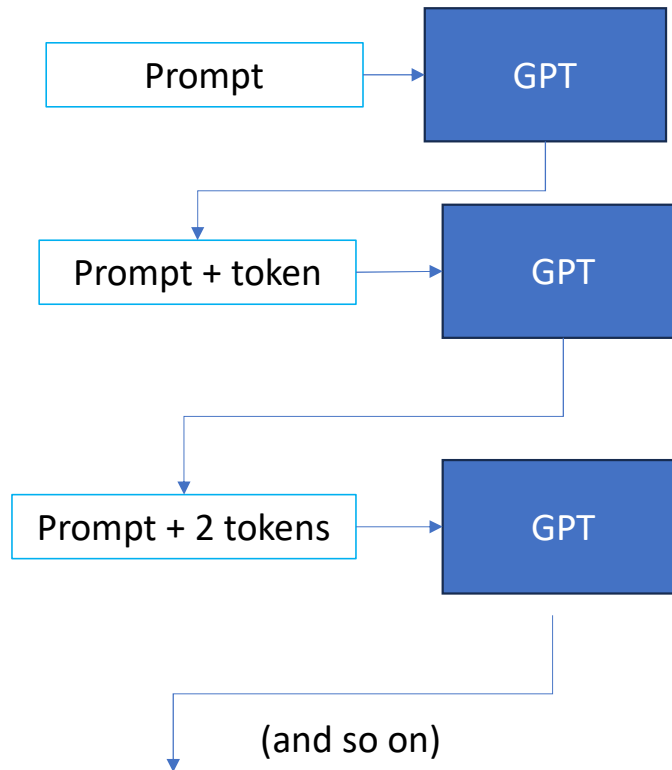
Sequence to sequence: if you actually need to generate output text

Remember the encoder-decoder architecture:



Encoder/decoder alternative:

When a GPT model outputs text, it probably does something like this:



Some practical considerations:

Encoding special tokens:

Generally, when we wish to encode-decode sentences, we reserve a particular set of tokens for signalling the ends of the sentence. We call these tokens the SOS (start of sentence) and EOS (end of sentence). This will make it much easier, in particular for an RNN network, to know what is going on.

Sequence padding:

Sentences have different lengths. All the sequence models we have dealt with so far can easily work with that. However, when we train our network, we will usually put multiple pieces of text in our mini batches. Due to how tensors are designed, those inputs will have to have the same length (in whatever embedding you are using), otherwise they can't be in the same minibatch.

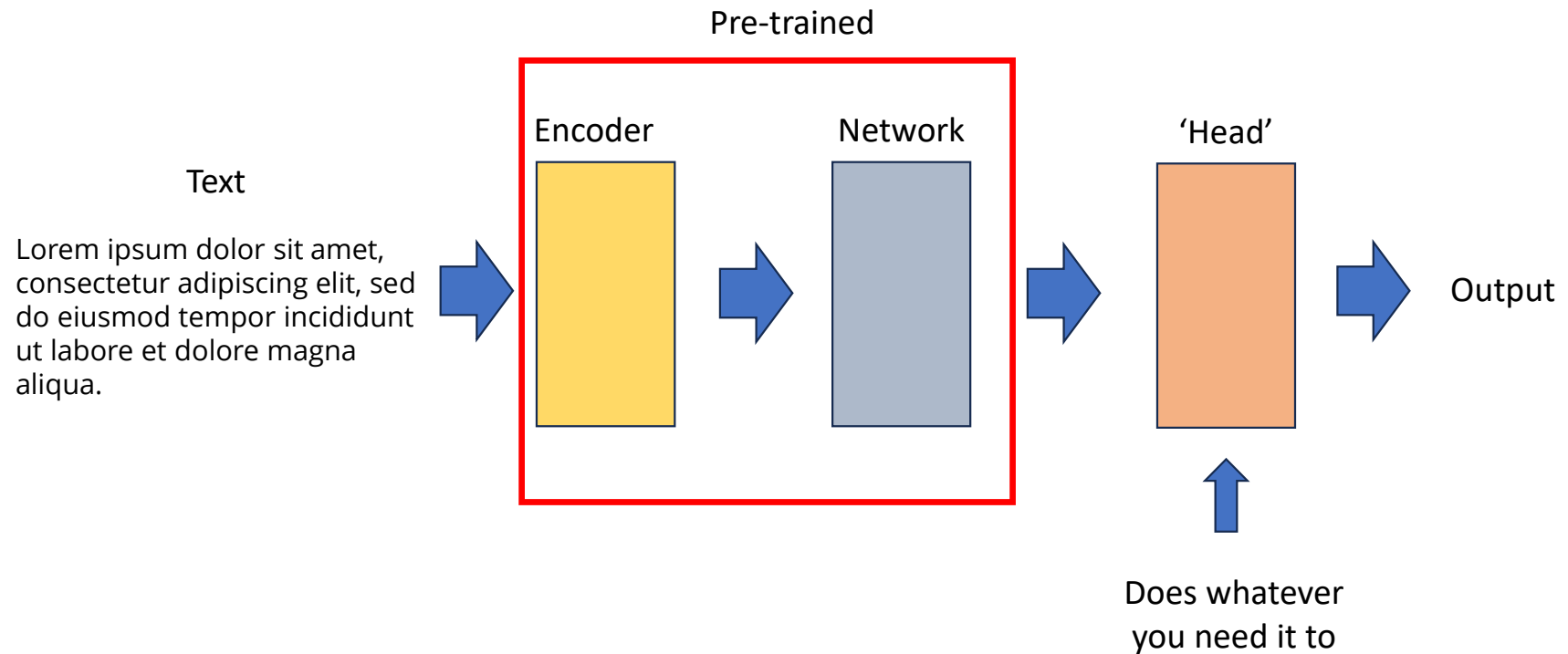
'Vectorizing':

It is almost never a good idea to simply convert text to scalar representations. This means, if you have 'encoded' your text using a dictionary, such that each letter/word/phoneme/whatever is represented as a single scalar, you should still one-hot encode those scalars to make them into vectors. It makes the job a lot easier for your model.



Fine tuning

The picture from before



Popular pre-trained models (open source):

GPT-2: <https://openai.com/research/gpt-2-1-5b-release>

Falcon: <https://falconllm.tii.ae/falcon-models.html>

Meta's Llama: <https://ai.meta.com/llama/>

Note that both Falcon and Llama exist in very large sizes (Llama up to 405B parameters). A sensible workflow would be to start with one of the smaller sizes, get everything to work, and then figure out if you have access to a cloud service where the big models can be fine tuned.

In the course project, you would probably have plenty to work with simply using the smallest models.

<https://huggingface.co/>

I have no idea what the story behind the logo/name is...

But: 'huggingface' has become the one-stop place to go if you want to put together an NLP pipeline in python. And most of the tools are written for pytorch, so it is all very easy.

Huggingface has everything from pretrained models, to tutorials, to datasets, embeddings and advanced pipeline-building tools.



chatGPT & Prompt engineering

Within this course, 'prompt engineering' is not deep learning. Meaning that you can not pass the final by simply prompt-engineering your way through chatGPT.

However, it is interesting to discuss how prompt engineering relates to the structured data that chatGPT was trained on. (more at the end of the slide show)



Reading

Required watching:

BERT inventor talks about BERT: <https://www.youtube.com/watch?v=knTc-NQSjKA>

Optional reading:

OpenAI's GPT-2: <https://jalammar.github.io/illustrated-gpt2/> (extremely good explanation of both GPT-2 and GPT-3, and transformers in general)

Same guy explaining GPT-3 (in less detail): <https://jalammar.github.io/how-gpt3-works-visualizations-animations/>

Minimal GPT implementation in pytorch: <https://github.com/karpathy/minGPT>

Paper presenting GPT-3: <https://arxiv.org/abs/2005.14165>

Interesting investigation of model scaling in NLP: <https://arxiv.org/pdf/2001.08361.pdf>



Exercises

Embeddings:

Play around with embeddings and embedding vectors

(<https://colab.research.google.com/drive/1cksiYvKZpgt2KAd6E3pXei8UxXYXAtrE?usp=sharing>)

Date translation:

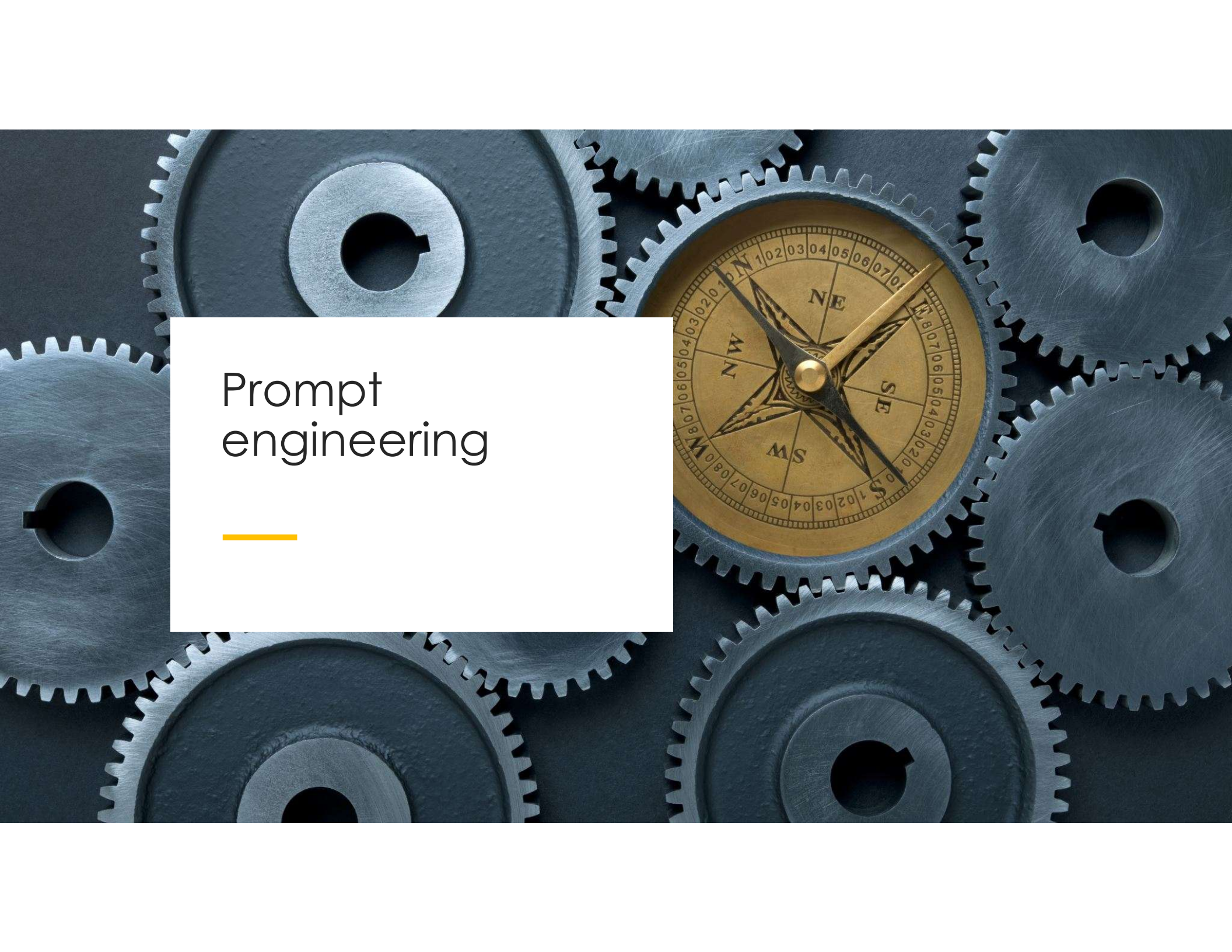
https://colab.research.google.com/drive/151f3Er6k8gG0kQufkeB8TAB_aPyJJWik?usp=sharing

GPT-2:

Trying a pretrained version of GPT-2 from huggingface, to generate new text from a prompt:

<https://colab.research.google.com/drive/1gtGK9wzkPRauUNuwVgv1S16UE18jV3du?usp=sharing>





Prompt engineering

Playing around with the chatGPT API:

For this presentation, I decided to implement an example with the openAI API.

I decided to make a set of functions that could open a folder of python code and tell me what was going on in there.

First thing: registering with openAI and getting a user key.



How do prompts work?

Chat-GPT tries to continue text.

If the beginning of a text is a request for an answer, a logical continuation is that answer.

If the prompt also contains examples of what an answer would look like, then such an answer becomes a much more plausible continuation.

First I asked GPT-3.5 to describe each file:

```
def describe_file(filename):
    baseString='Please describe the following python file. In the description, first describe
    which other files are called by this file, and the describe the contents of the file. Please
    output a description of the file, using at most 100 words. The file in question is: '

    baseString=baseString + ''' + filename + ''' + ': (start of file)          ' +
    open(filename).read() + ' (end of file).'
    completion = openai.ChatCompletion.create(
        model="gpt-3.5-turbo-16k",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": baseString},
        ]
    )
    return completion['choices'][0]['message']['content']

descriptions=[]
for i in range(len(newList)):
    descriptions.append(describe_file(newList[i]))
```



Then I got it to combine all descriptions:

```
baseString='In the following text, multiple files from the directory ' + codeFolder + ' are described. Please combine the descriptions into a single description of the directory, and summarize which files are called most. Please output both a tabular description of the whole directory, and a summary of the most important files, using at most 500 words for the summary. The files in question are: '
```

```
for i in range(len(newList)):
    baseString=baseString + ', ' + descriptions[i] + ', '
```

```
completion = openai.ChatCompletion.create(
    model="gpt-3.5-turbo-16k",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": baseString},
    ]
)
```

```
completion['choices'][0]['message']['content']
```



Directory: ../EEGprediction

Name	Number of Calls	Description
compareModels.py	0	Imports various modules and libraries, loads a pickled model, defines functions and variables
data_utils2.py	3	Imports libraries, defines functions and variables
data_utils4.py	1	Imports modules and libraries, defines functions and a class
EEG_ERP_1.py	0	Imports modules and libraries, processes EEG data, calculates ERP, and visualizes results
EEG_ERP_pred2.py	1	Imports modules and libraries, loads a neural network model, preprocesses EEG data, predicts ERP
lightningTransformer3.py	1	Imports various standard libraries, defines classes and functionality for the transformer model
main2_gaussianPred.py	2	Imports modules and libraries, defines functions, sets device and paths, trains a model, and plots results
main3_transformer_withFiltering.py	2	Imports modules and libraries, defines functions, sets up the GPU device, trains a model, and plots results
main4_linearAgain.py	1	Imports modules and libraries, defines functions, sets device and paths, trains a linear regression model
main5_linearClean.py	1	Imports the "data_utils2.py" file, defines functions, sets paths, creates datasets, trains a linear model
oracleTiming.py	0	Imports modules and libraries, defines functions, loads training and validation data, and trains a model
parameterTest.py	0	Calls various files, defines classes, generates and loads data, and trains a neural network
playground.py	0	Imports functions and defines a function for returning file paths, demonstrates usage of the files
rasmus_train.py	0	Imports modules and libraries, defines functions, performs hyperparameter tuning, and trains a model

Based on the descriptions, the files with the most calls by other files are:

- data_utils2.py: 3 calls
- main2_gaussianPred.py: 2 calls
- main3_transformer_withFiltering.py: 2 calls

Prompt engineering!

So, what might we do to improve this?

Well, we can give the machine a better idea of what each description should look like, so I changed the prompt a little bit:

```
'...Let the description be at most 25 words for each file. An example of a  
file description is: "Defines a neural network class called "neural  
network" with associated helper functions". Another example is: "trains a  
neural network using data from folder XX"...'
```



New output:

File Name	Number of Calls	Description
compareModels.py	0	-
data_utils2.py	5	Defines helper functions and data preprocessing.
data_utils4.py	3	Contains utility functions and an EEG dataset class.
EEG_ERP_1.py	1	Preprocesses EEG data, calculates ERP, and visualizes the results.
EEG_ERP_pred2.py	1	Preprocesses EEG data, predicts ERP components, and visualizes the results.
lightningTransformer3.py	1	Implements a transformer model using PyTorch Lightning for EEG prediction.
main2_gaussianPred.py	1	Creates a dataset, predicts using Gaussian Process Regressor, and plots the result
main3_transformer_withFiltering.py	1	Trains a transformer model for EEG prediction, with filtering and logging.
main4_linearAgain.py	1	Trains and evaluates a linear regression model for EEG prediction.
main5_linearClean.py	1	Fits and evaluates a Linear Regression model for EEG prediction.
oracleTiming.py	1	Loads data, creates a neural network model, and trains it using PyTorch Lightning
parameterTest.py	1	Defines two classes for testing neural networks.
playground.py	1	Interacts with MNE-BIDS library to read and process data from a BIDS dataset.
rasmus_train.py	1	Trains a deep learning model using PyTorch Lightning and performs hyperparameter t

One last trick, make it list the calling functions first, and count that list:

File Name	Number of Calls	Description
compareModels.py	0	Imports various libraries, loads a pickled model, defines functions, performs tests
data_utils4.py	3	Imports various modules, defines several functions and a class for EEG dataset management
EEG_ERP_1.py	0	Processes EEG data, calculates ERP, and visualizes the results.
EEG_ERP_pred2.py	0	Calls various files, loads a neural network, preprocesses EEG data, predicts ERP
lightningTransformer3.py	3	Implements a transformer-based model using PyTorch Lightning with various classes
main2_gaussianPred.py	0	Implements various helper functions, predicts using Gaussian Process Regressor, and
main3_transformer_withFiltering.py	0	Trains a transformer model for EEG prediction with filtering and implements logging
main4_linearAgain.py	0	Imports necessary libraries, trains, tests, and evaluates a linear regression model
main5_linearClean.py	0	Implements processes to train, test, and evaluate a Linear Regression model with
oracleTiming.py	0	Imports various modules, defines functions, loads data, creates a neural network
parameterTest.py	0	Contains two classes, creates an instance of one class, loads data, and trains a
playground.py	0	Interacts with MNE-BIDS library to read and process BIDS dataset data. Demonstrates
rasmus_train.py	0	Calls various files, trains a deep learning model, performs hyperparameter tuning

Cost:

Time from starting to learn about the API until writing previous slide: ~ 6 working hours

Cost of calling the API in this time: \$0.13 (USD)

